On-Device Indic Voice Cloning and Text-to-Speech Synthesis for Edge Architectures

Mid-Semester Project Report

Submitted in partial fulfillment of the requirements of
CS F377 Design Project

By

Sudhanva B H
ID No. 2024A7PS0085H

Under the supervision of:

Dr. Jabez Christopher

Reshma A.S.



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, HYDERABAD CAMPUS
October 2026

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, HYDERABAD CAMPUS

Abstract

On-Device Indic Voice Cloning and Text-to-Speech Synthesis for Edge Architectures

by Sudhanva B H

On-device zero-shot neural voice cloning enables localized replication of an unseen human speaker's acoustic timbre and prosody from a brief reference recording (three to five seconds) without fine-tuning or server communication. This report investigates this capability to assist government school classrooms and visually impaired students with accessible audio learning in their local teacher's voice. In foundational education, familiarity with an instructor's voice offers compelling pedagogical benefits for learner comfort and engagement, while visually impaired students depend heavily on spoken instruction for curriculum access. Performing inference entirely on-device avoids the need to transmit biometric voice recordings to external inference services, reducing network-related privacy exposure.

Deploying deep generative speech models on classroom edge hardware introduces severe operational constraints: rural schools frequently lack reliable connectivity, have zero budget for cloud API fees, and utilize subsidized tablets such as the Lenovo Tab K10 (MediaTek Helio P22T with 4 GB RAM). To evaluate edge feasibility, we first implement a mobile proof of concept (PoC) using ZipVoice-Distill (123M parameters, INT8 ONNX graphs) with a 24 kHz Vocos vocoder executed via Sherpa-ONNX inside a Flutter Android prototype. Physical hardware benchmarking on the Lenovo Tab K10 confirms operational viability with zero *LowMemoryKiller* terminations across nine trials, yielding mean Real-Time Factors of 18.22× (short), 10.78× (medium), and 11.20× (long utterances), and post-utterance resident memory readings reaching up to 2,491 MB.

These empirical observations highlight the need for careful memory budgeting and latency reduction on resource-constrained devices, as well as the linguistic limitations of existing English-centric edge models. Guided by these insights, we engineer our primary Indic speech synthesis pipeline around a 22-layer Diffusion Transformer (IndicF5, 335M parameters). We establish a calibrated syllabic duration heuristic (6.5 mel frames per Unicode character, ≈ 69.3 ms per character), export INT8-quantized ONNX graphs (reducing DiT weights from 1.35 GB to 349 MB and Vocos to 22 MB) yielding an analytical memory budget of 801 MB, and develop a native C++ runtime interface. Finally, we formulate a concrete future roadmap targeting sampling step distillation and Google LiteRT, outlining the path toward efficient multilingual edge voice cloning.

Acknowledgements

I am grateful to Prof Dr. Jabez Christopher for providing me the opportunity to work under him as part of the Design Project. I appreciate his invaluable guidance and support throughout the course of the project. His expertise in the AI domain and insightful feedback has led to the success of this work. I am filled with gratitude to receive his mentorship which has been essential for my personal and professional development. I would also like to extend my gratitude to Reshma A.S. for her valuable guidance and planning throughout the project.

Contents

Abstract i

Acknowledgements ii

Contents iii

List of Tables vii

List of Figures viii

1 Introduction and Problem Formulation 1
- 1.1 Teacher-Voice Learning in Government Schools and Accessibility for Blind Students 1
  - 1.1.1 Voice Identity, Pedagogical Trust, and Biometric Privacy 2
  - 1.1.2 Evolution of Speech Synthesis Paradigms 2
- 1.2 The Zero-Shot In-Context Voice Cloning Paradigm 3
- 1.3 Rationale for On-Device Edge Execution 4
- 1.4 The Tri-Factor Challenge in Educational Edge Deployment 5
- 1.5 Embedded Hardware Constraints and Operating System Resource Envelopes . . 5
  - 1.5.1 Android Memory Management and the LowMemoryKiller (LMK) 6
- 1.6 Evaluation Framework for Edge Speech Generative Systems 6
  - 1.6.1 Physical Memory Confinement and Resident Memory 7
  - 1.6.2 Storage Footprint and Package Compressibility 7
  - 1.6.3 Acoustic and Speaker Identity Transfer Fidelity 7
  - 1.6.4 Computational Throughput and RTF 7
  - 1.6.5 Characterizing Short-Utterance Latency and Throughput Scaling 8
- 1.7 Scope and Structural Organization of the Report 8

2 Theoretical Foundations and Comprehensive Literature Survey 10
- 2.1 Taxonomy of Deep Generative Speech Architectures 10
  - 2.1.1 Autoregressive Acoustic Models 10
  - 2.1.2 Variational Autoencoders and Normalizing Flows 11
  - 2.1.3 Score-Based Diffusion Generative Models 11
  - 2.1.4 Continuous Flow Matching Principles 12
- 2.2 Continuous Normalizing Flows and Optimal Transport Flow Matching 12
  - 2.2.1 Conditional Flow Matching (CFM) 12
  - 2.2.2 Optimal Transport (OT) Displacement Interpolant 13
- 2.3 Diffusion Transformers (DiT) and Non-Autoregressive Sequence Modeling 13

2.3.1 Backbone Microarchitecture . . . 13
2.3.2 Rotary Position Embeddings (RoPE) . . . 14
2.3.3 ConvNeXt-V2 Text Representation and Fused Input Conditioning . . . 14
2.4 In-Context Speech Inpainting and Acoustic Timbre Transfer . . . 14
2.5 Neural Vocoding Paradigms: Transposed Convolutions vs. Fourier Synthesis . . . 15
2.5.1 The HiFi-GAN Paradigm and Edge Bottlenecks . . . 15
2.5.2 Vocos: Fourier Magnitude and Phase Synthesis . . . 16
2.6 Numerical Solvers and Trajectory Dynamics . . . 16
2.6.1 Euler Numerical Integration . . . 17
2.6.2 Sway Sampling Parameterization . . . 17
2.7 Trajectory Straightening and Step Distillation for Edge Efficiency . . . 17
2.7.1 Rectified Flow and Reflow (Liu et al., 2022) . . . 18
2.7.2 FlashSpeech and Consistency Distillation (Ye et al., 2024) . . . 18
2.8 Indic Linguistic Phonetics and Acoustic Corpora . . . 18
2.8.1 The Akshara Syllabic Structure . . . 19
2.8.2 Acoustic Duration Properties (IndicVoices) . . . 19
2.8.3 IndicF5 and Multilingual Adaptation (AI4Bharat, 2025) . . . 19
2.9 Edge Machine Learning Runtimes: Runtime Comparative Analysis . . . 20
2.9.1 Sherpa-ONNX . . . 20
2.9.2 ONNX Runtime Mobile (C++ / Dart FFI) . . . 21
2.9.3 Google LiteRT (Formerly TensorFlow Lite) . . . 21

3 System Architecture and Algorithmic Formulation 22
3.1 System Architecture: Architectural Progression from Proof of Concept to Production Pipeline . . . 22
3.2 Signal Preprocessing and Acoustic Feature Extraction . . . 24
3.2.1 Standardized Audio Ingestion and Format Transcoding . . . 24
3.2.2 Root-Mean-Square (RMS) Energy Normalization . . . 24
3.2.3 Log-Mel Filterbank Transformation . . . 25
3.3 Indic Akshara Dynamics and Syllabic Duration Modeling . . . 26
3.3.1 The Linguistic Duration Mismatch Problem . . . 26
3.3.2 Mathematical Derivation of the Calibrated Syllabic Model . . . 26
3.4 Conditioning Representation and Sequence Concatenation . . . 28
3.4.1 Multilingual Character Tokenization . . . 28
3.4.2 The Spatial Alignment Constraint (Reference Duration Law) . . . 28
3.5 Continuous Flow Matching Sampling Formulation . . . 29
3.6 Vocos Neural Spectral Decoding via Inverse STFT . . . 29
3.7 Formal Algorithmic Specifications . . . 30
3.8 Comparative Model Architectural Matrix . . . 30

4 Implementation Methodology and Step-by-Step Engineering Iterations 32
4.1 Iterative Engineering Methodology . . . 32
4.2 Iteration 1: Mobile Feasibility Proof of Concept (ZipVoice-Distill via Sherpa-ONNX) . . . 32
4.2.1 Objective: Testing Edge Flow-Matching Feasibility . . . 32
4.2.2 Implementation Details . . . 33
4.2.3 Findings and Milestones . . . 33

4.3 Iteration 2: Flutter Mobile PoC Prototype (`voice_clone_proto`) 33
4.3.1 Objective and UI Architecture 33
4.3.2 System Implementation 34
4.3.3 Concurrency and Background Isolate Decoupling 34
4.4 Iteration 3: Multi-Device Benchmarking of the PoC and Empirical Inferences . . 34
4.4.1 Benchmarking Methodology 34
4.4.2 Empirical Hardware Measurements 35
4.4.3 Analysis of PoC Benchmarks and Inferences for IndicF5 and Google LiteRT 37
4.5 Iteration 4: Transition to IndicF5: Multilingual Adaptation and Desktop Verification 38
4.5.1 Objective: Realizing the PoC Inferences for Indian Languages 38
4.5.2 Model Setup and Adaptation 38
4.5.3 Validation Across Indian Languages 38
4.6 Iteration 5: Syllabic Duration Calibration (Resolving Audio Truncation and Trailing Silence) 39
4.6.1 The Duration Mismatch Issue 39
4.6.2 Root Cause Investigation 39
4.6.3 Resolution: Calibrated 6.5 Mel Frames per Character 39
4.7 Iteration 6: Sequence Alignment and Reference Audio Duration Constraints . . . 40
4.7.1 The Long-Reference Degradation Issue 40
4.7.2 Root Cause Investigation 40
4.7.3 Resolution: Reference Duration Recommendation 41
4.8 Iteration 7: In-Memory Warm Model Batch Synthesis and Dataset Curation . . . 41
4.8.1 Motivation and Batch Engine 41
4.8.2 System Architecture 41
4.8.3 The Multilingual Dataset (`prompts.json`) 41
4.9 Iteration 8: Unified Voice Studio Suite and Dynamic Catalog Management 42
4.9.1 System Architecture 42
4.9.2 Dynamic Catalog Management (`samples/voices.json`) 42
4.10 Iteration 9: Graph Optimization and INT8 Quantization 43
4.10.1 Objective: Decoupling from the PyTorch Runtime 43
4.10.2 Exported Model Artifacts and Sequence Dimension Formulation 43
4.11 Iteration 10: Custom Native C++ Engine and Dart FFI Shared Library 44
4.11.1 Motivation 44
4.11.2 C++ Engine Architecture (`indicf5_engine.cpp`) 44
4.12 Hardware Memory Budget Analysis on 4 GB RAM Platforms 45
4.13 Evaluation Limitations and Future Validation Plan 46

5 Strategic Future Architecture: Google LiteRT Implementation and Edge Acceleration Roadmap 47
5.1 Motivation and Architectural Rationale for Transitioning to Google LiteRT . . . 47
5.2 Direct PyTorch Conversion via LiteRT Torch (`litert-torch`) 48
5.2.1 StableHLO Dialect and Operator Fusion 49
5.3 Hardware Acceleration via the CompiledModel API 49
5.3.1 Declarative Accelerator Prioritization 49
5.3.2 Qualcomm AI Engine Direct (QNN) Integration on Compatible Silicon . . 50

5.3.3 MediaTek Silicon Considerations and CPU-Side Optimization on Budget Platforms . . . 50
5.4 Zero-Copy Pipelines via Android Hardware Buffers . . . 51
5.5 Trajectory Distillation: 2-Step and 4-Step Sampling . . . 51
5.5.1 Rectified Flow Trajectory Straightening (4-Step Reflow) . . . 51
5.5.2 Consistency Distillation and Few-Step Trajectory Solving . . . 52
5.6 Physical Testing and Verification Protocol . . . 52

6 Conclusion and Research Summary **53**
6.1 Summary of Research Accomplishments and Milestones . . . 53
6.2 Key Scientific, Systems, and Societal Contributions . . . 54
6.2.1 Theoretical and Linguistic Contributions . . . 54
6.2.2 Systems and Edge Deployment Contributions . . . 54
6.2.3 Pedagogical and Accessibility Intent in Untapped Domains . . . 55
6.3 Ethics, Biometric Privacy, and Responsible Deployment . . . 55
6.4 Concluding Remarks and Immediate Priorities . . . 56

A Mathematical Derivations and Numerical Proofs **57**
A.1 Derivation of the Optimal Transport Flow Matching Objective . . . 57
A.1.1 Probability Paths and the Continuity Equation . . . 57
A.1.2 Gradient Equivalence of the Loss Objective . . . 58
A.2 Analytical Formulation of the Inverse STFT with Window Overlap-Add . . . 59

B Multilingual Phonetic Evaluation Prompts Dataset **61**
B.1 Dataset Architecture and Stratification Methodology . . . 61
B.2 Telugu Dataset Inventory . . . 62
B.3 Tamil Dataset Inventory . . . 63
B.4 Hindi Dataset Inventory . . . 64

Bibliography **65**

List of Tables

3.1 Architectural Comparison of Speech Synthesis Models 30

4.1 Empirical On-Device Benchmarks Across Physical Android Devices 36
4.2 Raw Experimental Observations for ZipVoice PoC on Lenovo Tab K10 36
4.3 Analytical Memory Budget for Proposed Quantized IndicF5 Deployment 46

5.1 Architecture Comparison: ONNX Runtime vs. Google LiteRT 48

B.1 Curated Telugu Evaluation Prompts across Standardized Tiers 62
B.2 Curated Tamil Evaluation Prompts across Standardized Tiers 63
B.3 Curated Hindi Evaluation Prompts across Standardized Tiers 64

List of Figures

3.1 End-to-End System Architecture Across Project Phases 24
3.2 IndicF5 Zero-Shot Speech Synthesis Architecture 27

4.1 Mean RTF Across Utterance Tiers on Lenovo Tab K10 37
4.2 Post-Utterance Resident Memory on Lenovo Tab K10 37
4.3 Native IndicF5 Runtime and Edge Deployment Architecture 45

5.1 Strategic Edge Acceleration and Optimization Roadmap 48

Chapter 1

Introduction and Problem Formulation

1.1 Teacher-Voice Learning in Government Schools and Accessibility for Blind Students

In primary and upper-primary government schools across India, millions of children undertake foundational education in regional languages such as Telugu, Tamil, and Hindi. Educational research suggests that young learners can benefit from higher cognitive engagement, listening comprehension, and emotional comfort when instructional materials are spoken in the familiar, trusted voice of their classroom teacher rather than an unfamiliar or synthetic persona. While rigorous empirical validation through classroom user studies remains an ongoing objective, the potential pedagogical value of teacher-voiced instruction offers a strong motivation for accessible speech synthesis.

For visually impaired (blind) students, the importance of voice is even more profound. Auditory learning serves as their primary sensory medium for textbooks, practice exercises, and conceptual explanations. Hearing lessons delivered in their own teacher's voice provides pedagogical continuity, comfort, and cognitive trust, helping bridge the gap between classroom instruction and independent study.

Despite these compelling educational opportunities, this domain remains largely untapped by contemporary speech synthesis:

1. **Commercial Focus on Other Domains**: Commercial voice synthesis platforms primarily focus on Western languages, media dubbing, and commercial entertainment, leaving vernacular public school education and special-needs accessibility largely unaddressed.
2. **Offline Classroom Realities**: Many government schools operate in rural and semi-urban areas with intermittent or nonexistent internet access, making cloud-dependent speech synthesis unusable in everyday classrooms.

3. **Hardware Budgets**: State educational programs distribute budget Android tablets (such as the Lenovo Tab K10 with 4 GB RAM) with zero recurring budget for cloud API subscription fees.

4. **Biometric Privacy and Consent**: Public schools have a duty of care to ensure that the voice recordings of teachers and children remain under local control and are not transmitted to third-party cloud servers.

To address these needs, we explore an on-device, zero-shot multilingual voice cloning pipeline. Under this approach, an instructor records a single brief reference recording (three to five seconds) locally on the classroom tablet; the system can then synthesize textbook passages, interactive exercises, and audio aids for visually impaired learners in that teacher's voice, operating completely offline at zero recurring subscription cost.

1.1.1 Voice Identity, Pedagogical Trust, and Biometric Privacy

A human speech signal jointly conveys linguistic content, speaker identity, prosody, and acoustic environment characteristics within a few seconds of audio. In zero-shot voice cloning, the goal is to synthesize arbitrary new text in the voice of an unseen speaker using only a brief reference recording (three to five seconds), without modifying model parameters or running gradient updates during inference.

While frontier generative speech models achieve high perceptual naturalness, existing deployments operate almost exclusively on cloud servers equipped with high-end GPUs. This status quo presents three major obstacles for public education:

(i) **Biometric Privacy and Consent Considerations**: Transmitting unencrypted speech samples to external servers exposes teachers' voiceprints to data retention, misuse, and unauthorized synthesis risks. Performing inference locally ensures that acoustic biometric samples remain contained on the device, where local storage policies, explicit instructor consent, and immediate deletion controls can be enforced.

(ii) **Lack of Regional Language Coverage**: Over one billion people speak Dravidian and Indo-Aryan languages, which are the primary instructional media in state schools. Most commercial edge voice models support only English or Mandarin.

(iii) **Network Dependency**: In rural government classrooms and special education facilities, internet connectivity is often absent, rendering cloud-dependent educational assistants unusable.

1.1.2 Evolution of Speech Synthesis Paradigms

Automated Text-to-Speech (TTS) architectures have developed across three major generations:

1. *Formant synthesis* modeled the human vocal tract through systems of acoustic resonators. Although computationally lightweight, the generated audio sounded unnatural and metallic.

2. *Concatenative synthesis* assembled waveforms from multi-hour databases of pre-recorded speech fragments. This approach yielded high segmental naturalness, but required hundreds of megabytes of storage per voice, could not modify prosody dynamically, and required recording an entire dataset to support a new speaker.

3. *Statistical parametric speech synthesis* (SPSS) modeled acoustic features using Hidden Markov Models. While SPSS reduced storage requirements and permitted parametric voice transformations, over-smoothing often produced muffled output.

The introduction of deep neural networks led to end-to-end architectures such as Tacotron 2, FastSpeech, and Variational Inference with adversarial learning for End-to-end Text-to-Speech (VITS) [1], alongside neural vocoders such as WaveNet and High-Fidelity Generative Adversarial Network (HiFi-GAN) [2], which achieved human-like Mean Opinion Scores (MOS). Nevertheless, traditional neural pipelines remain ill-suited for mobile deployment: they require minutes of clean audio for speaker adaptation, and their multi-stage architectures generally demand high-power GPUs.

Modern zero-shot in-context voice cloning models address this limitation by conditioning generation directly on short reference clips without gradient updates. This investigation adapts and optimizes that paradigm for on-device deployment across Indian languages.

1.2 The Zero-Shot In-Context Voice Cloning Paradigm

Formally, zero-shot voice cloning denotes the capacity of an acoustic model to synthesize arbitrary target text in the vocal identity, pitch range, and cadence of an unseen speaker, conditioned solely on a brief reference audio prompt (three to five seconds), without parameter fine-tuning.

Let $\mathbf{x}_{\text{ref}} \in \mathbb{R}^{C \times T_{\text{ref}}}$ denote the intermediate acoustic representation (such as a 100-channel log-mel spectrogram) extracted from a reference recording of duration T_{ref}, and let $\mathbf{w}_{\text{ref}} = (w_1^{\text{ref}}, w_2^{\text{ref}}, \ldots, w_{N_{\text{ref}}}^{\text{ref}})$ denote its linguistic transcript. Given a novel target text sequence $\mathbf{w}_{\text{tgt}} = (w_1^{\text{tgt}}, w_2^{\text{tgt}}, \ldots, w_{N_{\text{tgt}}}^{\text{tgt}})$, zero-shot speech synthesis models the conditional distribution:

$$p(\mathbf{x}_{\text{tgt}} \mid \mathbf{w}_{\text{tgt}}, \mathbf{x}_{\text{ref}}, \mathbf{w}_{\text{ref}}; \theta) \tag{1.1}$$

such that the synthesized target mel-spectrogram $\mathbf{x}_{\text{tgt}} \in \mathbb{R}^{C \times T_{\text{tgt}}}$ matches the text sequence $\mathbf{w}_{\text{tgt}}$ while replicating the acoustic identity of $\mathbf{x}_{\text{ref}}$.

Early zero-shot systems relied on speaker encoder networks trained with contrastive objectives (such as the generalized end-to-end loss). These encoders projected variable-length audio into fixed-dimensional speaker embeddings (d-vectors or x-vectors) $\mathbf{e}_{\text{spk}} \in \mathbb{R}^D$, typically with $D \in$

$[256, 512]$. The acoustic model was conditioned on $\mathbf{e}_{\text{spk}}$ via feature concatenation or adaptive instance normalization (AdaIN) layers.

However, global embedding vectors introduce an information bottleneck: a single static vector struggles to preserve fine-grained temporal cues, including prosodic variations, co-articulation patterns, and room reverberation characteristics.

To address this limitation, recent models such as Voicebox [3], Embarrassingly Easy Non-Autoregressive Text-to-Speech (E2 TTS) [4], and F5-TTS [5] frame zero-shot speech synthesis as an audio inpainting problem. Instead of compressing the reference signal into a single vector, continuous acoustic frames $\mathbf{x}_{\text{ref}}$ are concatenated directly with the target generation trajectory along the time axis. The model uses bidirectional self-attention mechanisms over the fused sequence representations to learn dependencies between the reference acoustics and the target phonetic sequence, harmonizing speaker timbre across the transition boundary without relying on separate cross-attention layers.

1.3 Rationale for On-Device Edge Execution: Privacy, Connectivity, and Operational Costs in Public Education

Although cloud-based voice cloning systems achieve strong fidelity on server GPUs, relying on remote infrastructure introduces three critical obstacles for public education:

1. **Biometric Data Protection and Local Control**: A human voice recording constitutes a sensitive biometric identifier. In public schools, uploading raw audio samples of teachers and students to external commercial servers creates privacy and governance concerns regarding data retention, unauthorized voice harvesting, and unconsented cloning. Performing inference offline ensures that acoustic data remains strictly within device memory, where transparent local access controls and informed teacher consent can be enforced.

2. **Reliable Offline Operation in Vernacular Classrooms**: In many regions of the Indian subcontinent, internet access in government primary and secondary schools remains intermittent or completely absent. In rural classrooms and special education facilities for visually impaired students, continuous network access cannot be assumed. Edge execution ensures deterministic, uninterrupted learning and audio study aid generation in complete airplane mode.

3. **Elimination of Cloud Infrastructure and Per-Character Costs**: Commercial speech synthesis APIs incur recurring per-minute and per-character fees driven by GPU server maintenance. Public schools and state education departments operate under strict budgetary boundaries with zero recurring budget for cloud subscriptions. Running models directly on subsidized student tablets eliminates external infrastructure costs entirely, enabling personalized educational audio generation without ongoing service fees.

1.4 The Tri-Factor Challenge in Educational Edge Deployment

Deploying continuous flow-matching models on budget mobile hardware in government classrooms exposes a multi-constraint optimization problem, which we designate as the Tri-Factor Challenge:

1. **Native Indic and Dravidian Phonological Support for State Curricula**: Over 1.4 billion people speak Dravidian and Indo-Aryan languages, which form the primary instructional medium in state board curricula (such as Telugu, Tamil, and Hindi). These languages feature distinct phonetic inventories, agglutinative morphology, syllabic akshara scripts, consonant conjuncts (*samyuktaksharas*), and dependent vowel signs (*matras*). Western and Chinese zero-shot models (e.g., Kokoro-82M, standard F5-TTS, or ZipVoice-Distill) lack Indic character vocabularies and phonetic coverage, making them incapable of synthesizing Indian school textbooks.
2. **Zero-Shot In-Context Generalization from Brief Classroom Samples**: The system must adapt to an unseen teacher from a single, three-to-five second recording captured in an ordinary classroom environment, without gradient updates or fine-tuning. Autoregressive models (e.g., XTTS-v2 at 467M parameters [6]) suffer from $O(T)$ sequential latency, quadratic attention memory, and hallucination loops. Non-autoregressive flow-matching models generate frames in parallel, but require low-step numerical solvers to remain feasible on mobile hardware.
3. **Strict Physical Edge Memory and Compute Budgets on Subsidized Tablets**: The pipeline must run entirely on-device without cloud connectivity. Commodity edge devices distributed in state education programs have limited RAM, bounded memory bandwidth, thermal constraints, and heterogeneous CPU clusters without dedicated NPUs.

Existing open-source and commercial models typically address at most two of these requirements: models that run on-device are limited to English and Chinese; models that support Indian languages rely on remote server clusters; and lightweight on-device engines are restricted to single-speaker statistical synthesis. This investigation targets the intersection of all three criteria to empower public education and accessibility.

1.5 Embedded Hardware Constraints and Operating System Resource Envelopes

To design an edge architecture capable of practical mass deployment across public schools, our engineering target is anchored against budget commodity hardware rather than flagship silicon. In this project, our primary reference hardware is the **Lenovo Tab K10**, a tablet widely procured and distributed in government school digitization programs across India, with the following specifications:

- **System-on-Chip (SoC)**: MediaTek Helio P22T (MT8768T), built on a 12 nm FinFET process.
- **Processor Microarchitecture**: An octa-core ARM Cortex-A53 cluster in a big.LITTLE configuration (4 cores up to 2.3 GHz and 4 efficiency cores up to 1.8 GHz). The Cortex-A53 implements an in-order execution pipeline with dual-issue capabilities, lacking modern out-of-order execution optimizations.
- **Vector Acceleration**: Standard ARMv8-A NEON 128-bit Single Instruction Multiple Data (SIMD) execution units.
- **Neural Processing Hardware**: Zero dedicated NPU or AI accelerator. All deep learning inference operations (matrix multiplications, convolutional projections, and Fourier transformations) execute on the CPU cluster.
- **Physical System Memory**: 3 GB or 4 GB LPDDR4X RAM over a 32-bit dual-channel memory bus with a theoretical peak bandwidth capped at approximately 14.9 GB/s.

1.5.1 Android Memory Management and the LowMemoryKiller (LMK)

Memory management on modern Android devices is mediated by kernel memory pressure notifications and the userspace low-memory killer daemon (`lmkd`). Unlike desktop operating systems that rely on swap-to-disk partitions to absorb arbitrary memory allocations, Android systems utilize in-memory compressed swap (ZRAM) to protect mobile flash storage (eMMC and UFS) from write-endurance degradation while compressing inactive memory pages.

When physical RAM pressure rises, `lmkd` terminates cached and background processes based on their Out-Of-Memory adjustment score (`oom_adj_score`). Foreground applications actively presenting a user interface enjoy high scheduling priority, but their memory consumption is constrained by system-wide memory pressure, device RAM capacity, and concurrent operating system services (such as SurfaceFlinger and system servers). On a 4 GB device, system services and framework processes typically occupy 2.0 to 2.5 GB of RAM. While foreground processes can temporarily reach higher memory usage (as observed in our empirical benchmarking where post-utterance resident memory reached up to 2,491 MB without termination), operating under sustained high resident memory increases vulnerability to termination under background multitasking pressure.

Consequently, to maintain robust process stability across diverse device conditions, an analytical memory optimization target of $\leq 1,000$ MB (with an engineering component budget of approximately 801 MB) represents a prudent design guideline for edge generative models.

1.6 Evaluation Framework for Edge Speech Generative Systems

To evaluate on-device speech synthesis, we define an evaluation framework balancing physical hardware constraints, vocal cloning fidelity, and inference throughput:

1.6.1 Physical Memory Confinement and Resident Set Size (RSS)

The core operational criterion for on-device viability is physical memory confinement. Memory consumption is quantified not by virtual address reservations (`VmSize`), which reflect mapped address space, but by Resident Set Size (`VmRSS`), which measures physical RAM allocated to the process by the kernel memory subsystem. Memory readings can be obtained by polling `/proc/self/status` upon completion of utterance generation. For sustained stability on 4 GB platforms, our analytical design target aims to keep resident memory well within available limits to avoid triggering system-wide memory reclaim pressure.

1.6.2 Storage Footprint and Package Compressibility

On consumer mobile hardware, application package size directly affects user adoption. Large desktop checkpoints requiring gigabytes of disk space are impractical for distribution. We evaluate models based on their quantized on-disk footprint, targeting ≤ 400 MB for the combined acoustic model and neural vocoder.

1.6.3 Acoustic and Speaker Identity Transfer Fidelity

Voice cloning fidelity is evaluated along two complementary dimensions:

- **Speaker Similarity and Naturalness**: Evaluated through subjective assessment such as Mean Opinion Scores (MOS) on a 5-point Likert scale, assessing how faithfully the synthesized audio replicates the timbre, pitch contour, and perceived identity of the reference speaker.
- **Acoustic Spectral Distortion**: Measured via Mel-Cepstral Distortion (MCD) between ground-truth and synthesized acoustic trajectories:

$$\mathrm{MCD} = \frac{10\sqrt{2}}{\ln 10} \frac{1}{T} \sum_{t=1}^{T} \sqrt{\sum_{d=1}^{D} (c_t[d] - \hat{c}_t[d])^2} \tag{1.2}$$

  where $c_t[d]$ and $\hat{c}_t[d]$ denote the d-th mel-frequency cepstral coefficients of target and synthesized frames.

1.6.4 Computational Throughput and Real-Time Factor (RTF)

Operational throughput is quantified via the RTF, defined as the ratio between the total wall-clock computational latency required to synthesize the audio and the physical duration of the resulting synthesized speech waveform:

$$\mathrm{RTF} = \frac{T_{\mathrm{synthesis}}}{T_{\mathrm{audio}}} = \frac{\Delta t_{\mathrm{proc}}}{\frac{N_{\mathrm{samples}}}{f_s}} \tag{1.3}$$

where Δt_{proc} represents execution time in seconds, N_{samples} denotes synthesized audio samples, and f_s represents the acoustic sampling frequency (24,000 Hz).

In edge environments lacking discrete GPUs, the immediate goal is to establish stable, deterministic execution latencies on mobile ARM CPUs, providing an empirical baseline for future runtime optimizations and hardware acceleration.

1.6.5 Characterizing Short-Utterance Latency and Throughput Scaling

In non-autoregressive flow-matching systems, empirical benchmarking across sentence lengths reveals that short utterances frequently exhibit elevated Real-Time Factors compared to longer passages. In traditional autoregressive models, latency scales approximately linearly with sequence length ($O(T)$). In continuous flow matching, total synthesis latency reflects a combination of fixed setup costs and length-dependent numerical integration:

$$T_{\text{synthesis}} = T_{\text{setup}} + N_{\text{steps}} \cdot T_{\text{forward}}(T_{\text{audio}}) + T_{\text{vocoder}}(T_{\text{audio}}) \tag{1.4}$$

Here, T_{setup} includes text parsing, vocabulary lookup, intermediate buffer preparation, and vocoder session setup. When synthesizing a brief utterance (such as a 1.2-second phrase), fixed setup and memory allocation costs constitute a substantial proportion of overall runtime, elevating the observed RTF. As utterance duration increases to medium ($T_{\text{audio}} \approx 5.5\,\text{s}$) and long ($T_{\text{audio}} \approx 15.0\,\text{s}$) sequences, fixed overhead is amortized across a longer generated duration, allowing throughput to approach the steady-state execution speed of the underlying neural solver. Dedicated stage-level profiling is necessary to isolate and quantify each constituent latency component precisely.

1.7 Scope and Structural Organization of the Report

This mid-semester technical report documents the theoretical foundations, iterative engineering milestones, hardware profiling results, and production roadmaps developed for on-device voice cloning. The remainder of this document is organized as follows:

- **Chapter 2 (Theoretical Foundations and Comprehensive Literature Survey)** provides a mathematical review of Continuous Normalizing Flows (CNF), Optimal Transport Flow Matching (CFM), Diffusion Transformers (DiT), neural vocoders (comparing HiFi-GAN and Vocos), trajectory straightening via Rectified Flow, step distillation via FlashSpeech, Dravidian/Indic phonology, and edge machine learning runtimes.
- **Chapter 3 (System Architecture and Algorithmic Formulation)** details the system architecture progressing from our mobile proof of concept to the primary IndicF5 edge architecture, the signal conditioning pipeline, the calibration of our syllabic akshara duration model, the Euler ODE numerical solver, and formal algorithmic specifications.

- **Chapter 4 (Implementation Methodology and Step-by-Step Engineering Iterations)** chronicles the ten engineering iterations executed during the project, detailing the empirical mobile proof of concept, physical hardware benchmarking across four silicon tiers, the inferences guiding our Indic implementation, the resolution of duration truncation and spatial alignment anomalies, integer quantization pipelines, and native C++ execution on mobile hardware.
- **Chapter 5 (Strategic Future Architecture: Google LiteRT Implementation and Edge Acceleration Roadmap)** establishes the technical blueprint for migrating the IndicF5 pipeline to Google LiteRT, outlining the `litert-torch` conversion flow, hardware accelerator targeting via the `CompiledModel` API (Qualcomm QNN HTP and MediaTek NeuroPilot), zero-copy buffer sharing, and 2-step distillation for mobile efficiency.
- **Chapter 6 (Conclusion and Research Summary)** summarizes the engineering accomplishments and outlines immediate next steps.
- **Appendices** provide formal mathematical derivations and an inventory of the curated multilingual conversational dataset.

Chapter 2

Theoretical Foundations and Comprehensive Literature Survey

2.1 Taxonomy of Deep Generative Speech Architectures

Neural generative speech synthesis models the probability distribution of acoustic representations conditioned on linguistic text and speaker descriptors. Modern deep generative paradigms partition into four foundational mathematical families:

2.1.1 Autoregressive Acoustic Models

Autoregressive (AR) architectures formulate speech generation as a sequential next-token prediction task. Representing the acoustic feature sequence as $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T)$, the joint probability is factored via the chain rule of probability:

$$p(\mathbf{X} \mid \mathbf{W}) = \prod_{t=1}^{T} p(\mathbf{x}_t \mid \mathbf{x}_{<t}, \mathbf{W}; \theta) \tag{2.1}$$

where $\mathbf{x}_{<t} = (\mathbf{x}_1, \ldots, \mathbf{x}_{t-1})$ represents the historical acoustic context and $\mathbf{W}$ denotes the conditioning text sequence. Architectures such as Tacotron, AudioLM, and XTTS-v2 [6] leverage causal multi-head self-attention mechanisms [7] to parameterize each conditional step.

While autoregressive models capture expressive prosody effectively, they present practical computational bottlenecks for edge devices:

(a) **Inference Complexity** $O(T)$: Generating T acoustic frames strictly requires T sequential forward passes through the network. On a mobile CPU, where each forward pass consumes 15–30 ms, generating a 10-second utterance ($T \approx 937$ frames) demands over 15 to 25 seconds of compute (RTF > 2.0).

(b) **Exposure Bias and Compounding Errors**: Because the model is trained with teacher-forcing but tested with its own predictions, small deviations at step t compound recursively over time, sometimes resulting in phoneme deletions, unnatural pitch inflections, or repetition loops.

(c) **KV-Cache Memory Overhead**: Retaining historical key-value tensors across deep transformer stacks inflates RAM usage linearly with sequence length, increasing Low Memory Killer (LMK) termination risks on 4 GB Android devices.

2.1.2 Variational Autoencoders and Normalizing Flows

Conditional variational autoencoders (VAEs) and finite normalizing flows (such as VITS [1]) model speech by maximizing the variational evidence lower bound (ELBO). By employing finite normalizing flows parameterized by stacks of invertible affine coupling layers, VITS establishes bijective mappings between an isotropic Gaussian latent prior and the target acoustic distribution. While VITS achieves non-autoregressive parallel synthesis ($O(1)$ forward passes) with fast execution speeds, its mathematical structure relies on Monotonic Alignment Search (MAS) and dynamic programming alignment paths. These aligners degrade in zero-shot cross-lingual transfer, as the internal phonetic duration predictors cannot accommodate unseen multilingual scripts without dedicated acoustic phoneme dictionaries.

2.1.3 Score-Based Diffusion Generative Models

Denoising Diffusion Probabilistic Models (DDPM) [8] and continuous score-based generative models [9] synthesize data by reversing a progressive forward noising process. Formally, a forward Stochastic Differential Equation (SDE) corrupts the data distribution $p_{\text{data}}(\mathbf{x})$ into an isotropic Gaussian distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$ via an Itô stochastic differential equation:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w} \tag{2.2}$$

where $\mathbf{w}$ is standard Brownian motion. Generation is achieved by solving the reverse-time SDE:

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\right] dt + g(t)d\bar{\mathbf{w}} \tag{2.3}$$

where $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the score function, approximated by a deep neural network $\mathbf{s}_\theta(\mathbf{x}, t)$.

Although diffusion models generate high-quality audio, their stochastic trajectories are inherently curved. Simulating stochastic differential equations requires numerical solvers across 50 to 1,000 sampling steps. Even accelerated diffusion solvers (such as DPM-Solver) require at least 15 to 30 forward evaluations, incurring unviable latency on mobile ARM processors.

2.1.4 Continuous Flow Matching Principles

To avoid the curved trajectories and stochastic overhead of diffusion SDEs, Lipman et al. [10] formulated *Continuous Flow Matching* (CFM). CFM synthesizes data along deterministic probability trajectories governed by ordinary differential equations. By constructing linear conditional displacement interpolants, CFM ensures that individual paths between prior noise and clean speech follow straight lines, providing a sound theoretical framework for low-step numerical solving on resource-constrained edge architectures.

2.2 Continuous Normalizing Flows and Optimal Transport Flow Matching

Let $\mathbb{R}^d$ denote the data space representing flattened mel-spectrogram tensors. A Continuous Normalizing Flow (CNF) defines a time-dependent push-forward transformation $\psi_t : \mathbb{R}^d \to \mathbb{R}^d$ for $t \in [0, 1]$, driven by a time-dependent vector field $\mathbf{v}_t : \mathbb{R}^d \to \mathbb{R}^d$ via the initial value problem (ODE):

$$\frac{d}{dt}\psi_t(\mathbf{x}) = \mathbf{v}_t(\psi_t(\mathbf{x})), \quad \psi_0(\mathbf{x}) = \mathbf{x} \tag{2.4}$$

The vector field induces a continuous probability density path $p_t(\mathbf{x})$ satisfying the continuity equation:

$$\frac{\partial p_t(\mathbf{x})}{\partial t} + \nabla \cdot (p_t(\mathbf{x})\mathbf{v}_t(\mathbf{x})) = 0 \tag{2.5}$$

where $p_0(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mathbf{0}, \mathbf{I})$ represents the tractable standard Gaussian prior, and $p_1(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$ denotes the empirical target speech distribution.

2.2.1 Conditional Flow Matching (CFM)

Directly regressing the marginal vector field $\mathbf{v}_t(\mathbf{x})$ is computationally intractable because the marginal density $p_t(\mathbf{x}) = \int p_t(\mathbf{x} \mid \mathbf{x}_1)q(\mathbf{x}_1)d\mathbf{x}_1$ involves an integral over the entire data manifold. Lipman et al. [10] proved that regressing conditional vector fields $\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)$ conditioned on individual data samples $\mathbf{x}_1$ yields identical parameter gradients:

Theorem 2.1 (Equivalence of CFM and Marginal Objectives [10])**.** *Let $p_t(\mathbf{x} \mid \mathbf{x}_1)$ be a conditional probability path generated by conditional vector field $\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)$. Define the marginal vector field $\mathbf{v}_t(\mathbf{x}) = \int \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\frac{p_t(\mathbf{x}|\mathbf{x}_1)q(\mathbf{x}_1)}{p_t(\mathbf{x})}d\mathbf{x}_1$. Then the objective:*

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t\sim U[0,1],\mathbf{x}_1\sim q(\mathbf{x}_1),\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{x}_1)} \left\|\mathbf{v}_\theta(\mathbf{x}, t) - \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\right\|^2 \tag{2.6}$$

satisfies $\nabla_\theta \mathcal{L}_{CFM}(\theta) = \nabla_\theta \mathcal{L}_{Marginal}(\theta)$.

2.2.2 Optimal Transport (OT) Displacement Interpolant

Among the family of conditional probability paths connecting p_0 to p_1, pairing prior noise samples $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ with target mel-spectrograms $\mathbf{x}_1 \sim q(\mathbf{x}_1)$ via independent couplings yields the linear displacement interpolant:

$$\psi_t(\mathbf{x}_0) = (1 - (1 - \sigma_{\min})t)\mathbf{x}_0 + t\mathbf{x}_1 \tag{2.7}$$

where $\sigma_{\min} \approx 10^{-5}$ is an infinitesimal stability regularizer. Setting $\sigma_{\min} \to 0$ yields the canonical linear trajectory:

$$\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1 \tag{2.8}$$

Differentiating Equation 2.8 with respect to time t gives the target conditional velocity:

$$\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0, \mathbf{x}_1) = \frac{d\mathbf{x}_t}{dt} = \mathbf{x}_1 - \mathbf{x}_0 \tag{2.9}$$

The target velocity vector for any individual sample pair is constant across t. The neural network $v_\theta(\mathbf{x}_t, t, \mathbf{c})$ is trained to predict the displacement vector $(\mathbf{x}_1 - \mathbf{x}_0)$ given the intermediate state $\mathbf{x}_t$, time step t, and conditioning context $\mathbf{c}$:

$$\mathcal{L}_{\text{OT-CFM}}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{x}_1} \left\| \mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}) - (\mathbf{x}_1 - \mathbf{x}_0) \right\|_2^2 \tag{2.10}$$

It is important to note that while conditional trajectories for specific pairs $(\mathbf{x}_0, \mathbf{x}_1)$ are straight lines, the marginal velocity field $\mathbf{v}_t(\mathbf{x})$ learned by the neural network integrates over all possible couplings and can retain residual curvature. Consequently, straight conditional paths do not automatically guarantee that generation will succeed in arbitrarily few integration steps without empirical step tuning or trajectory straightening techniques.

2.3 Diffusion Transformers (DiT) and Non-Autoregressive Sequence Modeling

To parameterize the velocity network $\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c})$, recent architectures have adopted *Diffusion Transformers* (DiT) [5, 11] in place of traditional U-Net convolutional backbones.

2.3.1 Backbone Microarchitecture

In IndicF5, the velocity field estimator is structured as a 22-layer Diffusion Transformer with the following specifications:

- **Transformer Depth**: $L = 22$ identical transformer blocks.
- **Model Hidden Dimension**: $D = 1024$.

- **Attention Multi-Head Configuration**: $H = 16$ attention heads, yielding a per-head dimension of $d_h = \frac{D}{H} = 64$.
- **Feed-Forward Multiplier**: $\text{mult}_{\text{FFN}} = 2$, establishing an intermediate MLP expansion dimension of $D_{\text{FFN}} = 2048$.
- **Total Parameter Footprint**: Approximately 335 million parameters.

2.3.2 Rotary Position Embeddings (RoPE)

Unlike sinusoidal positional encodings that inject absolute coordinates into the input layer, IndicF5 employs *Rotary Position Embeddings* (RoPE) at every self-attention layer. Given a query vector $\mathbf{q}_m$ and key vector $\mathbf{k}_n$ at sequence positions m and n, RoPE applies an orthogonal rotation matrix $\mathbf{R}^d_{\Theta,m}$:

$$\tilde{\mathbf{q}}_m = \mathbf{R}^d_{\Theta,m}\mathbf{q}_m, \quad \tilde{\mathbf{k}}_n = \mathbf{R}^d_{\Theta,n}\mathbf{k}_n \tag{2.11}$$

such that their inner product encodes relative positional displacement:

$$\langle \tilde{\mathbf{q}}_m, \tilde{\mathbf{k}}_n \rangle = \mathbf{q}_m^T \mathbf{R}^d_{\Theta,n-m}\mathbf{k}_n = g(\mathbf{q}_m, \mathbf{k}_n, m-n) \tag{2.12}$$

This property enables the DiT to generalize to variable-length audio sequences during inference without positional coordinate degradation.

2.3.3 ConvNeXt-V2 Text Representation and Fused Input Conditioning

Text tokens are mapped to a 512-dimensional embedding space. Rather than passing text embeddings to separate cross-attention layers, IndicF5 processes the character token sequence through four ConvNeXt-V2 convolutional blocks incorporating depthwise convolutions, layer normalization, inverted bottlenecks, and Global Response Normalization (GRN). This extracts hierarchical local phonological representations.

The resulting text representation, the noisy mel tensor $\mathbf{x}_t$, and the reference acoustic prefix $\mathbf{x}_{\text{cond}}$ are concatenated along the channel dimension ($[\mathbf{x}_t \parallel \mathbf{x}_{\text{cond}} \parallel \mathbf{e}_{\text{text}}]$) and linearly projected into the transformer embedding dimension. The entire composite sequence is then processed through bidirectional self-attention blocks, enabling fused, single-stream representation modeling across linguistic and acoustic modalities.

2.4 In-Context Speech Inpainting and Acoustic Timbre Transfer

Modern zero-shot voice cloning architectures frame speaker identity transfer as an in-context audio inpainting problem [3, 4].

Let $\mathbf{x}_{\text{ref}} \in \mathbb{R}^{C \times T_{\text{ref}}}$ denote the mel-spectrogram extracted from a reference audio clip, and let $\mathbf{w}_{\text{ref}}$ represent its transcript. The model constructs a composite conditioning tensor along the

temporal dimension:

$$\mathbf{x}_{\text{cond}} = \left[\mathbf{x}_{\text{ref}} \,\middle\|\, \mathbf{0}_{C\times T_{\text{tgt}}}\right] \in \mathbb{R}^{C\times(T_{\text{ref}}+T_{\text{tgt}})} \tag{2.13}$$

where $\|$ denotes concatenation along the time axis, and $\mathbf{0}_{C\times T_{\text{tgt}}}$ represents zero-padding across the target generation window.

During training, random spans of audio are masked. The model learns to reconstruct the masked acoustic frames conditioned on the unmasked frames and the complete text sequence $[\mathbf{w}_{\text{ref}} \,\|\, \mathbf{w}_{\text{tgt}}]$.

During inference, the reference audio $\mathbf{x}_{\text{ref}}$ is placed in the unmasked prefix region, while the target region is initialized with Gaussian noise $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Throughout the numerical ODE integration, the velocity network v_θ reads the acoustic context in the prefix frames via bidirectional self-attention. The model extracts formant structure, glottal excitation timbre, speaking rate, and room acoustics from $\mathbf{x}_{\text{ref}}$, propagating these characteristics into the synthesized target frames $\mathbf{x}_{\text{tgt}}$ without requiring explicit speaker classification losses or static d-vector bottlenecks.

2.5 Neural Vocoding Paradigms: Transposed Convolutions vs. Fourier Synthesis

Once the mel-spectrogram tensor $\mathbf{x}_{\text{tgt}} \in \mathbb{R}^{C\times T_{\text{tgt}}}$ is synthesized by the flow-matching backbone, a neural vocoder reconstructs the continuous time-domain speech waveform $y(t) \in \mathbb{R}^{N_{\text{samples}}}$.

2.5.1 The HiFi-GAN Paradigm and Edge Bottlenecks

HiFi-GAN [2] has served as a widely adopted neural vocoder. It operates entirely in the time domain, employing a multi-receptive field fusion (MRF) module driven by a cascade of transposed convolutions to upsample 100-channel mel frames (hop size 256) to 24 kHz raw audio.

While HiFi-GAN produces high-quality audio, its design presents substantial computational overhead on mobile ARM processors:

1. **Memory Bandwidth Overhead**: Upsampling audio by factors of $8 \times 8 \times 2 \times 2 = 256$ requires repeatedly allocating and writing large intermediate feature buffers, exceeding mobile L1/L2 caches.
2. **Strided Access Patterns**: Multi-branch dilated convolutions induce strided memory access patterns, hindering continuous SIMD vectorization on ARM ARM NEON Advanced SIMD (NEON) units.

2.5.2 Vocos: Fourier Magnitude and Phase Synthesis

To avoid the physical bottlenecks of transposed convolutions, Siuzdak [12] developed Vocos. Rather than predicting raw time-domain samples through iterative upsampling, Vocos frames audio generation as complex spectral coefficient prediction.

Because the Short-Time Fourier Transform of a real-valued time-domain signal possesses Hermitian symmetry ($X(-f) = X^*(f)$), the full frequency spectrum is completely determined by $N_{\text{fft}}/2 + 1$ non-redundant frequency bins (513 bins for $N_{\text{fft}} = 1024$). The Vocos backbone consists of ConvNeXt blocks operating directly at the mel frame rate without temporal upsampling, predicting two spectral outputs:

1. **Magnitude Head**: Predicts non-negative magnitude spectra $|\mathbf{X}(f,t)| \in \mathbb{R}^{(N_{\text{fft}}/2+1)\times T}$.
2. **Phase Spectrogram Head**: Predicts unit complex numbers representing spectral phase angles $\phi(f,t) \in [-\pi, \pi]$ via two output channels $(\cos\phi, \sin\phi)$, normalized such that $\cos^2\phi + \sin^2\phi = 1$.

The complex Short-Time Fourier Transform (STFT) representation is assembled via:

$$\mathbf{X}(f,t) = |\mathbf{X}(f,t)| \odot (\cos\phi(f,t) + j\sin\phi(f,t)) \tag{2.14}$$

The time-domain audio waveform $y(t)$ is reconstructed analytically via the Inverse Short-Time Fourier Transform (iSTFT):

$$y(n) = \text{iSTFT}(\mathbf{X}(f,t)) = \frac{\sum_m \mathbf{x}_m(n - mH)w(n - mH)}{\sum_m w^2(n - mH)} \tag{2.15}$$

where $H = 256$ is the hop size and $w(n)$ is a Hanning synthesis window.

Because the Fourier inversion (Equation 2.15) is computed using optimized Fast Fourier Transform (SIMD) routines (such as ARM NEON FFT or vDSP), Vocos eliminates transposed convolutions entirely. On mobile ARM CPUs, Vocos runs substantially faster than HiFi-GAN while occupying an INT8 model footprint of ≈ 22 MB (13.5M parameters, compressed from 54 MB in FP32).

2.6 Numerical Solvers and Trajectory Dynamics

During inference, sampling from the velocity field $\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c})$ requires solving the initial value problem:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}), \quad \mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad t \in [0,1] \tag{2.16}$$

2.6.1 Euler Numerical Integration

Discretizing the time domain into N intervals, the first-order Euler update step is:

$$\mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \Delta t \cdot \tilde{\mathbf{v}}_\theta(\mathbf{x}_t, t, \mathbf{c}) \tag{2.17}$$

where $\tilde{\mathbf{v}}_\theta$ incorporates Classifier-Free Guidance (CFG) [13] with guidance weight $\gamma = 2.0$:

$$\tilde{\mathbf{v}}_\theta(\mathbf{x}_t, t, \mathbf{c}) = \mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}) + \gamma\left(\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}) - \mathbf{v}_\theta(\mathbf{x}_t, t, \emptyset)\right) = (1+\gamma)\mathbf{v}_\theta(\mathbf{x}_t, t, \mathbf{c}) - \gamma\mathbf{v}_\theta(\mathbf{x}_t, t, \emptyset) \tag{2.18}$$

Evaluating Equation 2.18 requires evaluating conditioned and unconditioned inputs (which can be batched together into a single $[2, C, T]$ forward pass).

2.6.2 Sway Sampling Parameterization

While uniform time steps $s_k = \frac{k}{N}$ represent a common discretization, the curvature of the learned vector field is non-uniform across the interval $[0, 1]$. To concentrate sampling steps where vector field trajectories require finer resolution, F5-TTS implements Sway Sampling [5] parameterized by coefficient $c \in [-1, 1]$:

$$t(s) = s + c \cdot \left(\cos\left(\frac{\pi}{2}s\right) - 1 + s\right), \quad s \in [0, 1] \tag{2.19}$$

When setting the sway coefficient to $c = -1.0$, this transformation simplifies to:

$$t(s) = 1 - \cos\left(\frac{\pi}{2}s\right) \tag{2.20}$$

Under this schedule, the derivative $\frac{dt}{ds} = \frac{\pi}{2}\sin\left(\frac{\pi}{2}s\right)$ is smallest near $s = 0$, clustering sampling evaluations more densely in the early generation stages where gross acoustic formant structure is established. In practice, sway sampling aids acoustic stability with fewer solver steps, though rigorous perceptual evaluations remain necessary to establish speech quality across diverse step counts.

2.7 Trajectory Straightening and Step Distillation for Edge Efficiency

To minimize iterative numerical evaluations, prevent battery depletion, and mitigate processor thermal throttling during edge execution on mobile SoCs, two acceleration methodologies have been proposed in the literature:

2.7.1 Rectified Flow and Reflow (Liu et al., 2022)

While optimal transport continuous flow matching produces straight trajectories in expectation, individual sample trajectories can cross in high-dimensional space, forcing the neural network to learn an average velocity that introduces curvature. Liu et al. [14] proved that this curvature can be systematically eliminated through recursive Reflow:

Definition 2.2 (The Reflow Operation [14])**.** Given a 1-rectified flow model v_{θ_1} trained on empirical pairs $(\mathbf{x}_0, \mathbf{x}_1)$, generate synthetic pairs $(\mathbf{x}_0, \mathbf{z}_1)$ by numerically integrating $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to $t = 1$ using v_{θ_1}. The 2-rectified flow model v_{θ_2} is trained on these synthetic straight-line pairs:

$$\mathcal{L}_{\text{Reflow}}(\theta_2) = \mathbb{E}_{\mathbf{x}_0, \mathbf{z}_1, t} \left\| \mathbf{v}_{\theta_2}((1-t)\mathbf{x}_0 + t\mathbf{z}_1, t) - (\mathbf{z}_1 - \mathbf{x}_0) \right\|^2 \tag{2.21}$$

By training on coupled pairs generated by the flow itself, trajectory intersections are untangled. The resulting vector field trajectories become straight lines, permitting numerical integration in as few as 4 Euler steps without perceptual degradation.

2.7.2 FlashSpeech and Consistency Distillation (Ye et al., 2024)

FlashSpeech [15] applies latent consistency models to non-autoregressive speech synthesis. Built upon an adversarial consistency training formulation, FlashSpeech trains a student network to predict the final acoustic representation directly while enforcing consistency between predictions evaluated at adjacent time steps along probability trajectories:

$$f_\theta(\mathbf{x}_{t_1}, t_1) = f_\theta(\mathbf{x}_{t_2}, t_2) \quad \forall\, t_1, t_2 \in [0, 1] \tag{2.22}$$

This self-consistency property collapses the multi-step numerical solving loop into one or two forward evaluations. While FlashSpeech was originally formulated for a distinct latent-consistency speech backbone rather than continuous flow matching, applying consistency distillation principles to IndicF5's DiT architecture constitutes an attractive proposed future direction to enable few-step inference on resource-constrained mobile hardware.

2.8 Indic Linguistic Phonetics and Acoustic Corpora

Indian languages belong predominantly to two distinct linguistic families: the Dravidian family (Tamil, Telugu, Kannada, Malayalam) and the Indo-Aryan family (Hindi, Marathi, Gujarati, Bengali, Punjabi, Odia, Assamese). Unlike English, which employs a Latin alphabet requiring non-deterministic grapheme-to-phoneme conversion (e.g., the pronunciation difference of "ough" in *through*, *rough*, and *thought*), Indic scripts are strictly phonetic and syllabic.

2.8.1 The Akshara Syllabic Structure

In Indic orthography, the core phonetic unit is the *akshara*. An akshara is an orthographic syllable structured as:

$$\text{Akshara} = C_0^k V \tag{2.23}$$

representing zero or more consonants (C) followed by an inherent vowel (V, typically /a/). Consonants are modified by dependent vowel diacritics (*matras*), vowel suppressors (*viramas* or *halant*), and nasalization marks (*anusvaras*). When multiple consonants combine without intervening vowels, they form composite consonant conjuncts (*samyuktaksharas*).

2.8.2 Acoustic Duration Properties (IndicVoices)

The IndicVoices speech corpus [16] comprises 1,417 hours of verified spontaneous and read speech recorded across 22 Indian languages. Acoustic analysis of this corpus demonstrates that an average Indic akshara possesses an acoustic duration spanning:

$$\Delta t_{\text{akshara}} \approx 150\,\text{ms} \quad \text{to} \quad 220\,\text{ms} \tag{2.24}$$

In digital mel-spectrogram representations utilizing a 24 kHz sampling rate and a hop size of 256 samples, each mel frame spans:

$$\Delta t_{\text{frame}} = \frac{256}{24000} \approx 10.667\,\text{ms} \tag{2.25}$$

Because an orthographic akshara typically decomposes into an average of 1.6 to 2.2 Unicode code points (base consonant plus vowel diacritics), dividing the composite syllabic duration (Equation 2.24) by code point count yields an average temporal density of approximately 65 to 75 ms per character token. At a frame interval of 10.667 ms, this corresponds to approximately 6.0 to 7.0 mel frames per Unicode character code point (with $k_{\text{mel}} = 6.5$ frames representing $\approx$ 69.33 ms). This relationship provides the empirical basis for the character-level duration heuristic calibrated in Chapter 3.

2.8.3 IndicF5 and Multilingual Adaptation (AI4Bharat, 2025)

IndicF5, formulated by Praveen et al. [17] at AI4Bharat (IIT Madras) in their work *"Phir Hera Fairy: An English Fairytaler is a Strong Faker of Fluent Speech in Low-Resource Indian Languages"*, adapts non-autoregressive continuous flow matching to the multilingual domain of the Indian subcontinent.

The authors evaluated whether an English-pretrained flow-matching checkpoint (F5-TTS [5]) could generalize to phonologically distinct Indian languages, testing three optimization strategies:

1. **Training from Scratch**: Initializing the 22-layer DiT backbone with randomized weights on Indian corpora. This yielded slow phonological convergence and unstable mel trajectories due to data scarcity in low-resource languages.

2. **Joint Multilingual Pre-training**: Fine-tuning simultaneously on paired English and Indic datasets. While stable, this experienced language competition and reduced prosodic expression in regional dialects.

3. **Exclusive Indic Fine-Tuning of the English Prior**: Fine-tuning the pre-trained English F5-TTS checkpoint exclusively on Indic speech data. This strategy achieved the highest naturalness (MOS) and speaker similarity, demonstrating that flow-matching vector fields learned from dense English acoustic priors provide effective inductive biases that transfer across phonetic boundaries without catastrophic forgetting.

Trained on 1,417 hours of multi-speaker speech sourced from IndicVoices-R [16], Rasa, IndicTTS, and LIMMITS across eleven official languages (Assamese, Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Tamil, and Telugu), IndicF5 demonstrates cross-lingual zero-shot cloning. A reference prompt in one language (such as Kannada or Odia) can guide synthesis in another language (such as Hindi or Tamil) while maintaining vocal timbre, pitch trajectory, and cadence. Furthermore, by defining a unified character tokenizer spanning 187 Unicode glyphs across Indic scripts and Latin characters, the model eliminates external G2P rule tables by learning character-to-acoustic alignments end-to-end.

2.9 Edge Machine Learning Runtimes: Runtime Comparative Analysis

Deploying deep flow-matching models on mobile devices requires an execution engine capable of parsing ONNX or FlatBuffer graphs, executing quantized integer operators, and offloading tensor operations to hardware accelerators. Three primary frameworks exist in the modern mobile ML ecosystem:

2.9.1 Sherpa-ONNX

Developed by the Next-gen Kaldi team [18], `sherpa-onnx` is an embedded C++ speech recognition and synthesis engine with Flutter/Dart bindings. Sherpa-ONNX provides specialized C++ loaders for Zipformer, VITS, and Matcha-TTS architectures. However, Sherpa-ONNX currently lacks native C++ loaders for Diffusion Transformers (DiT) and F5-TTS, preventing direct execution of IndicF5.

2.9.2 ONNX Runtime Mobile (C++ / Dart FFI)

Microsoft's ONNX Runtime Mobile [19] provides a generic C++ execution engine supporting standard ONNX graphs. It includes the XNNPACK execution provider for multi-threaded ARM NEON CPU acceleration and optional NNAPI/QNN delegates. However, executing custom multi-step ODE flow-matching loops requires developing custom C++ wrappers, shared libraries (`.so`), and manual Dart FFI bindings.

2.9.3 Google LiteRT (Formerly TensorFlow Lite)

Google LiteRT [20] is designed for mobile ML deployments, offering:

- Direct PyTorch conversion via `litert-torch` [21], avoiding intermediate ONNX operator translation issues.
- A unified `CompiledModel` API supporting automatic accelerator selection:
  - Qualcomm Hexagon HTP via Qualcomm QNN [22].
  - MediaTek APU via NeuroPilot [23].
  - Multi-core ARM NEON fallback via XNNPACK.
- Zero-copy hardware buffer management (`AHardwareBuffer`), minimizing memory transfers.

The architectural trade-offs across these runtimes are detailed in Chapter 3 and form the core of our future roadmap in Chapter 5.

Chapter 3

System Architecture and Algorithmic Formulation

3.1 System Architecture: Architectural Progression from Proof of Concept to Production Pipeline

To realize robust zero-shot voice cloning on resource-constrained Android hardware for government schools, the engineering methodology followed an evidence-driven architectural progression. Rather than deploying an unproven 335M parameter multilingual Diffusion Transformer directly to mobile devices without prior hardware characterization, we structured the investigation into three sequential phases:

1. **Phase 1: Mobile Feasibility Proof of Concept (PoC) (ZipVoice-Distill)**:
    - *Motivation*: To determine whether non-autoregressive continuous flow-matching speech synthesis could execute within mobile latency and memory budgets before undertaking complex Indic adaptation.
    - *Backbone Architecture*: A non-autoregressive Zipformer acoustic backbone comprising approximately 123 million parameters, pre-trained on the Emilia dataset.
    - *Numerical Solver*: A 4-step distilled continuous flow-matching decoder.
    - *Neural Vocoder*: A 24 kHz Vocos vocoder (54 MB FP32 graph, or 22 MB in INT8).
    - *Execution Runtime*: Executed natively via `sherpa-onnx` C++ binaries wrapped in a background Flutter Dart Isolate (`voice_clone_proto`).
    - *Model Assets*: An INT8 encoder (`encoder.int8.onnx`, $\approx$ 5.5 MB) and INT8 Zipformer decoder (`decoder.int8.onnx`, $\approx$ 124.7 MB), totaling $\approx$ 130.2 MB of acoustic ONNX graphs.
2. **Empirical PoC Findings and Inferences**: Physical hardware benchmarking of this PoC across Android silicon tiers (detailed in Chapter 4) demonstrated that flow-matching

synthesis successfully runs on client mobile processors, completing all experimental trials on the budget Lenovo Tab K10 without LowMemoryKiller termination. The PoC yielded four vital engineering inferences:

- *Linguistic Limitation*: ZipVoice is strictly restricted to English and Chinese orthography. It cannot process Dravidian and Indo-Aryan syllabic scripts (such as Telugu, Tamil, and Hindi) required for state school curricula.
- *Memory Variability and Allocation Pressure*: While all trials completed without process termination, post-utterance resident memory reached up to 2,491 MB on the Lenovo Tab K10, showing substantial allocation variance between examples. This demonstrated the necessity of strict memory budgeting and pre-allocated static arenas for long-term multi-turn stability.
- *Short-Utterance Latency Overhead*: Elevated Real-Time Factors on short phrases highlighted that unamortized setup and buffer allocations affect short-form generation, underscoring the need for warm in-memory pipelines.
- *Runtime Inflexibility*: Sherpa-ONNX provided C++ loaders for Zipformer, but lacked built-in support for Diffusion Transformers, requiring a custom native C++ engine for Indic models.

3. **Phase 2: Primary Indic Production Engine (IndicF5)**: Directly addressing the limitations exposed by the PoC, our primary system centers on IndicF5, an in-context flow-matching engine capable of authentic zero-shot voice cloning across Indian languages:
   - *Backbone Architecture*: A 22-layer Diffusion Transformer (DiT) comprising 335 million parameters, pre-trained on 1,417 hours of multi-speaker speech across eleven major Indian languages.
   - *Generative Formulation*: Continuous Flow Matching (CFM) solved via an 8-step Euler ODE solver with Classifier-Free Guidance ($\gamma = 2.0$) and trigonometric sway scheduling ($c = -1.0$).
   - *Neural Vocoder*: An INT8-quantized 24 kHz Vocos vocoder (22 MB, 13.5M parameters) computing analytical iSTFT acoustic waveforms.
   - *Analytical Memory Budget*: Static INT8 quantization compresses the DiT graph from 1.35 GB down to 349 MB, which, combined with pre-allocated static tensor arenas (280 MB) and runtime allocations (150 MB), establishes an analytical component-level memory budget of 801 MB.
   - *Execution Runtime*: Python desktop validation, a fixed-shape INT8 ONNX optimization prototype, a dynamic-axis FP32 ONNX graph for variable-length inference, and a custom native C++ engine with Dart FFI bindings.
4. **Phase 3: Strategic Edge Acceleration via Google LiteRT**: Because CPU-only execution on budget in-order Cortex-A53 cores (such as those in the Lenovo Tab K10) incurs high latency (RTF $> 10\times$), achieving interactive near-real-time performance requires architectural step distillation and runtime optimization. As detailed in Chapter 5, our roadmap targets 2-step consistency distillation alongside Google LiteRT operator fusion

for CPU efficiency on the K10, and hardware delegate offloading (Qualcomm QNN HTP and MediaTek NeuroPilot) for compatible accelerator-equipped platforms.

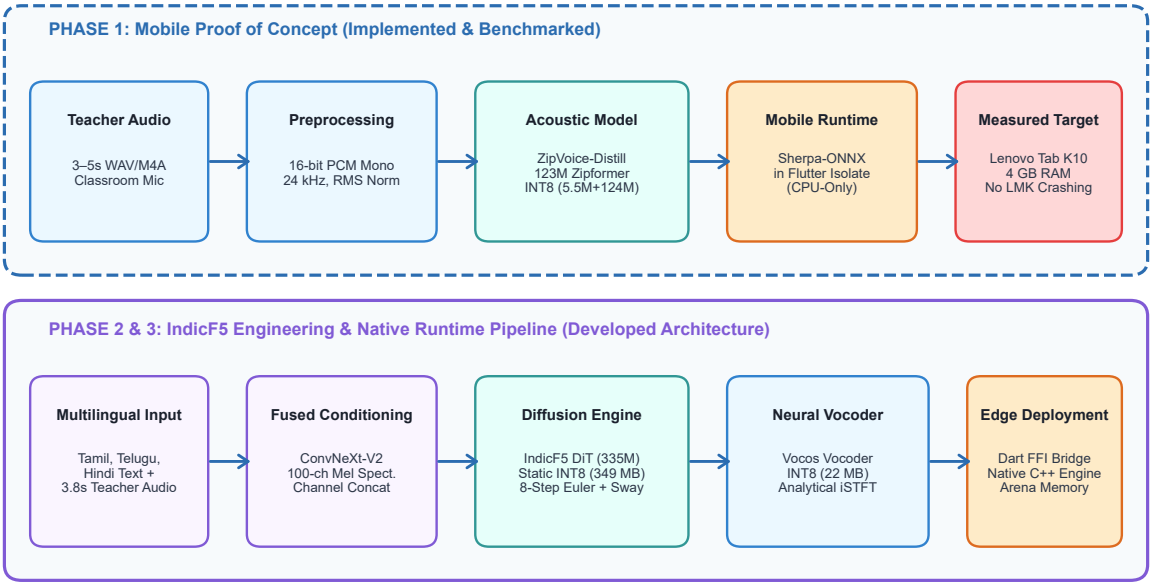


FIGURE 3.1: End-to-End System Architecture: Progression from the Implemented Mobile Proof of Concept (ZipVoice-Distill via Sherpa-ONNX) to the Primary IndicF5 Engineering Pipeline and Edge Deployment Architecture.

3.2 Signal Preprocessing and Acoustic Feature Extraction

Audio ingested from smartphone microphones or uploaded media containers exhibits variances in sampling rates, bit depths, channel configurations, and recording levels. To ensure numerical stability across the generative pipeline, every audio signal undergoes a standardized three-stage preprocessing pipeline:

3.2.1 Standardized Audio Ingestion and Format Transcoding

User-provided audio files (in `.mp3`, `.ogg`, `.m4a`, `.flac`, or `.wav` formats) are decoded and transcoded using FFmpeg into standardized 16-bit signed uncompressed Pulse Code Modulation (PCM) mono WAV files at a sampling frequency of:

$$f_s = 24,000\,\text{Hz} \tag{3.1}$$

3.2.2 Root-Mean-Square (RMS) Energy Normalization

Reference recordings exhibit noticeable dynamic range disparities, ranging from low-gain speech to saturated signals. In continuous flow matching, acoustic amplitude directly modulates the

magnitude of the conditioning mel frames $\mathbf{x}_{\text{ref}}$. If the reference audio energy is too low, the model predicts weak, muffled speech; conversely, excessive input gain introduces harmonic distortion and phase clipping.

To standardize acoustic energy, all reference audio signals $\mathbf{y} \in \mathbb{R}^N$ undergo Root-Mean-Square (RMS) normalization. The root-mean-square amplitude of the signal is defined as:

$$\text{RMS}(\mathbf{y}) = \sqrt{\frac{1}{N}\sum_{n=1}^{N} y(n)^2} \tag{3.2}$$

The audio waveform is scaled linearly to match a target reference energy level $\text{RMS}_{\text{target}} = 0.10$:

$$\mathbf{y}_{\text{norm}} = \mathbf{y} \cdot \left(\frac{\text{RMS}_{\text{target}}}{\text{RMS}(\mathbf{y}) + \epsilon}\right) \tag{3.3}$$

where $\epsilon = 10^{-7}$ prevents division by zero during silence. Following scaling, peak amplitude values are clamped to the dynamic range $[-1.0, 1.0]$ to prevent clipping:

$$\tilde{y}(n) = \max\left(-1.0, \min\left(1.0, y_{\text{norm}}(n)\right)\right) \tag{3.4}$$

3.2.3 Log-Mel Filterbank Transformation

Continuous acoustic features are extracted by computing 100-channel log-mel spectrograms. Let $N_{\text{fft}} = 1024$ represent the Fast Fourier Transform analysis window size, and let $H = 256$ represent the hop size. The Short-Time Fourier Transform (STFT) is computed using a periodic Hanning window:

$$X(k, m) = \sum_{n=0}^{N_{\text{fft}}-1} \tilde{y}(n + mH) w(n) e^{-j\frac{2\pi}{N_{\text{fft}}}kn} \tag{3.5}$$

where $k \in [0, \frac{N_{\text{fft}}}{2}]$ represents discrete frequency bins, and $m \in [0, M-1]$ denotes discrete time frames.

The physical power spectrum $|X(k, m)|^2$ is warped onto the perceptual mel scale via a bank of $C = 100$ triangular bandpass filters $H_c(k)$ spanning the frequency range $[0, \frac{f_s}{2}]$:

$$S_{\text{mel}}(c, m) = \sum_{k=0}^{N_{\text{fft}}/2} |X(k, m)|^2 H_c(k), \quad c \in [1, 100] \tag{3.6}$$

The log-mel spectrogram tensor $\mathbf{x}_{\text{mel}} \in \mathbb{R}^{100 \times M}$ is obtained via logarithmic compression:

$$\mathbf{x}_{\text{mel}}(c, m) = \log\left(\max\left(S_{\text{mel}}(c, m), 10^{-5}\right)\right) \tag{3.7}$$

3.3 Indic Akshara Dynamics and Syllabic Duration Modeling

A key challenge in non-autoregressive speech generation is determining the sequence length of the target audio prior to synthesis. Autoregressive models terminate generation organically upon emitting an end-of-sequence token. Non-autoregressive flow-matching models, by contrast, must pre-allocate a fixed-dimensional noise canvas of shape $[1, 100, T_{\text{total}}]$.

3.3.1 The Linguistic Duration Mismatch Problem

In English text-to-speech, character counts correlate loosely with duration due to irregular orthography, but phoneme counts provide reasonable estimates ($\approx$ 80–120 ms per phoneme). In Indian languages, written text consists of base consonants combined with dependent diacritics (*matras*, *viramas*, *anusvaras*).

Early IndicF5 implementations utilized an uncalibrated heuristic derived from English phoneme models:

$$T_{\text{gen}}^{\text{naive}} = \lfloor \text{len}(\mathbf{w}_{\text{tgt}}) \times 15.0 \rfloor \quad \text{frames} \tag{3.8}$$

This formula resulted in severe duration over-allocation. In Tamil, for example, an utterance containing 80 Unicode code points was allocated:

$$T_{\text{gen}} = 80 \times 15 = 1,200 \,\text{frames} \implies \frac{1200 \times 256}{24000} = 12.8 \,\text{seconds} \tag{3.9}$$

However, a native speaker articulates that same sentence in approximately 5.2 seconds (487 frames). Because the Diffusion Transformer was forced to integrate across a 1,200-frame canvas, the model articulated the speech correctly in the initial 5 seconds, but spent the remaining 7.6 seconds producing acoustic artifacts: audible breathing, murmuring sounds, repeated syllables, and background noise. Conversely, under-allocating duration resulted in truncated words, clipping the final words of sentences.

3.3.2 Mathematical Derivation of the Calibrated Syllabic Model

To resolve this issue, we derived a calibrated syllabic duration model grounded in the phonetic distributions of the IndicVoices corpus [16].

Let $N_{\text{chars}} = \text{len}(\mathbf{w}_{\text{tgt}})$ denote the Unicode string length of the target Indic text. In Dravidian and Indo-Aryan scripts, a complete syllabic akshara decomposes into an average of 1.6 to 2.2 Unicode code points (such as base consonant plus matra). Converting the hop interval ($H = 256$, $f_s = 24,000$ Hz) gives a mel frame step of $\Delta t_{\text{frame}} = \frac{256}{24000} \approx 10.667$ ms.

Allocating an empirical constant of $k_{\text{mel}} = 6.5$ mel frames per Unicode character code point yields an acoustic duration per code point of:

$$\tau_{\text{char}} = k_{\text{mel}} \cdot \Delta t_{\text{frame}} = 6.5 \times \frac{256}{24000} \,\text{s} \approx 69.33 \,\text{ms per character code point} \tag{3.10}$$

Across an entire orthographic akshara (1.6 to 2.2 code points), this corresponds to an effective syllable duration of ≈ 111 to 153 ms, matching natural speaking cadence. For a conversational sentence with 80 Unicode code points, the calibrated allocation is:

$$T_{\text{gen}} = 80 \times 6.5 = 520\,\text{frames} \implies \frac{520 \times 256}{24000} \approx 5.55\,\text{seconds} \tag{3.11}$$

Adding an explicit post-utterance acoustic buffer $\tau_{\text{pad}} = 0.30$ seconds ($T_{\text{pad}} = 28$ frames) yields an output duration of ≈ 5.85 seconds, closely matching natural human articulation. It should be noted that this formulation operates as a practical character-level heuristic rather than a full linguistically segmented akshara duration model; comprehensive evaluation across varied dialects, complex conjunct clusters, and speaking rates remains an open area for future validation.

Incorporating an adjustable user speaking rate modifier $\alpha_{\text{speed}} \in [0.5, 2.0]$, the target frame duration is calculated as:

$$T_{\text{gen}} = \left\lfloor \text{len}(\mathbf{w}_{\text{tgt}}) \times \frac{6.5}{\max(\alpha_{\text{speed}}, 0.1)} \right\rfloor \tag{3.12}$$

$$T_{\text{pad}} = \left\lfloor \frac{\tau_{\text{pad}} \cdot f_s}{H} \right\rfloor = \left\lfloor \frac{0.30 \times 24000}{256} \right\rfloor = 28\,\text{frames} \tag{3.13}$$

$$T_{\text{total}} = T_{\text{ref}} + T_{\text{gen}} + T_{\text{pad}} \tag{3.14}$$

where $T_{\text{ref}} = \lfloor \frac{N_{\text{ref_samples}}}{H} \rfloor$ represents the exact mel frame length of the reference audio clip. This calibrated formulation (Equation 3.14) eliminates both tail hallucinations and phoneme truncation across test utterances.

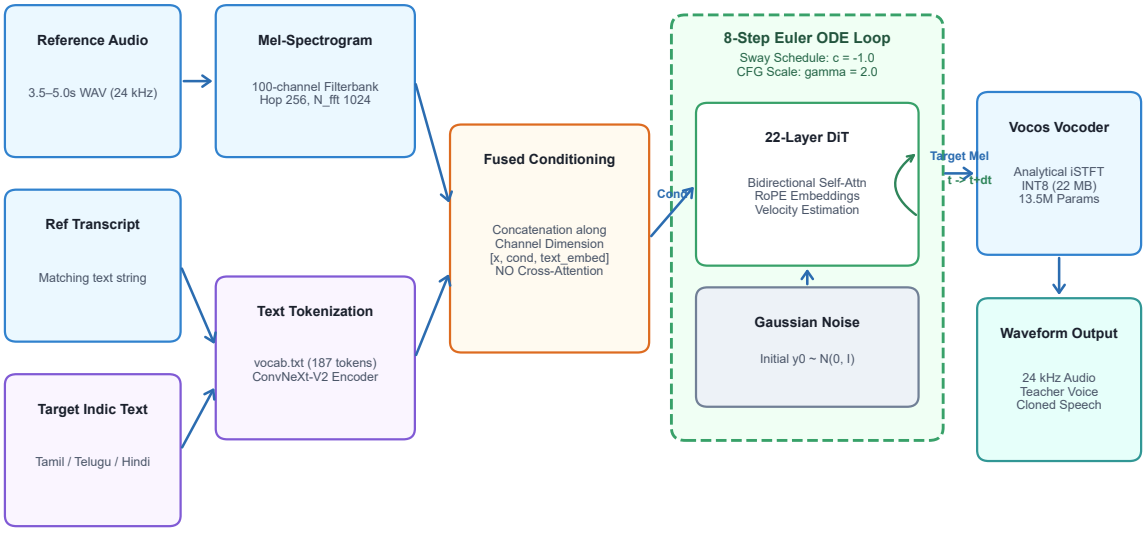


FIGURE 3.2: IndicF5 Zero-Shot Speech Synthesis Architecture: In-context reference conditioning, fused text and acoustic embedding, 8-step Euler ODE numerical integration with trigonometric sway time warping, and Vocos neural vocoding.

3.4 Conditioning Representation and Sequence Concatenation

In the F5-TTS and IndicF5 paradigm, zero-shot voice cloning operates without an explicit phoneme duration aligner or CTC loss. The architecture achieves monotonic alignment implicitly by concatenating text representations and acoustic frames along the sequence dimensions:

3.4.1 Multilingual Character Tokenization

Text sequences are parsed using a shared multilingual character vocabulary (`vocab.txt`) comprising 187 discrete tokens. The vocabulary encompasses:

- Latin alphabet characters (a–z, A–Z) for English baseline compatibility.
- Devanagari Unicode blocks (U+0900 to U+097F) for Hindi and Marathi.
- Tamil Unicode blocks (U+0B80 to U+0BFF) for Tamil.
- Telugu Unicode blocks (U+0C00 to U+0C7F) for Telugu.
- Kannada Unicode blocks (U+0C80 to U+0CFF) for Kannada.
- Standard punctuation markers (periods, commas, question marks, exclamation marks, and the Devanagari danda).

3.4.2 The Spatial Alignment Constraint (Reference Duration Law)

Let $\mathbf{w}_{\text{full}} = \mathbf{w}_{\text{ref}} \oplus \text{“ ”} \oplus \mathbf{w}_{\text{tgt}}$ represent the concatenated string. The text is passed through the ConvNeXt-V2 text encoder, producing a sequence of continuous text embeddings $\mathbf{E}_{\text{text}} \in \mathbb{R}^{512 \times N_{\text{tokens}}}$.

The model constructs an acoustic conditioning canvas $\mathbf{C} \in \mathbb{R}^{100 \times T_{\text{total}}}$:

$$\mathbf{C}[:, t] = \begin{cases} \mathbf{x}_{\text{ref}}[:, t], & \text{for } 0 \leq t < T_{\text{ref}} \\ \mathbf{0}, & \text{for } T_{\text{ref}} \leq t < T_{\text{total}} \end{cases} \tag{3.15}$$

Remark 3.1 (Reference Audio Duration Calibration). In the F5-TTS ConvNeXt architecture, the text encoder places text tokens contiguously at indices $0 \ldots N_{\text{tokens}}$ at the start of the sequence. If the reference audio duration T_{ref} is excessively long (for example, $T_{\text{ref}} > 7.0$ seconds), the text representations corresponding to the target utterance are positioned far behind the reference audio boundary. The self-attention layers suffer from positional degradation, attempting to synthesize speech across a temporal span where text conditioning is completely zero-padded, resulting in garbled output.

Conversely, if $T_{\text{ref}} < 3.0$ seconds, the reference signal provides insufficient acoustic context to extract stable speaker pitch contours and room reverberation profiles. Hence, we establish the

operational guideline:

$$3.5\,\text{seconds} \leq \Delta t_{\text{ref}} \leq 5.0\,\text{seconds} \tag{3.16}$$

accompanied by a verbatim transcript matching the spoken reference audio exactly.

3.5 Continuous Flow Matching Sampling Formulation

The generation trajectory is initialized at $t = 0$ by drawing Gaussian noise:

$$\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{100 \times T_{\text{total}}}) \tag{3.17}$$

Numerical integration proceeds from $s = 0$ to $s = 1$ across N discrete steps using the trigonometric sway-warped schedule:

$$s_i = \frac{i}{N}, \quad t_i = 1 - \cos\left(\frac{\pi}{2} s_i\right), \quad i \in \{0, 1, \ldots, N\} \tag{3.18}$$

with step size $\Delta t_i = t_{i+1} - t_i$. This transformation (derived from $t(s) = s + c \cdot (\cos(\frac{\pi}{2}s) - 1 + s)$ with $c = -1.0$) clusters sampling evaluations into the early stages of trajectory formation.

At each step i, the velocity vector field is evaluated twice to apply Classifier-Free Guidance (CFG):

$$\mathbf{v}_{\text{cond}} = \mathbf{v}_\theta(\mathbf{x}_{t_i}, t_i, \mathbf{C}, \mathbf{E}_{\text{text}}) \tag{3.19}$$

$$\mathbf{v}_{\text{uncond}} = \mathbf{v}_\theta(\mathbf{x}_{t_i}, t_i, \mathbf{0}, \mathbf{0}) \tag{3.20}$$

$$\tilde{\mathbf{v}}_i = (1 + \gamma)\mathbf{v}_{\text{cond}} - \gamma \mathbf{v}_{\text{uncond}} \tag{3.21}$$

where $\gamma = 2.0$ amplifies linguistic and speaker conditioning while suppressing unconditional background noise.

The state is updated via first-order Euler integration:

$$\mathbf{x}_{t_{i+1}} = \mathbf{x}_{t_i} + \Delta t_i \cdot \tilde{\mathbf{v}}_i \tag{3.22}$$

Upon reaching $t = 1$, the integrated trajectory $\mathbf{x}_1 \in \mathbb{R}^{100 \times T_{\text{total}}}$ represents clean mel-spectrogram features. The target speech frames are extracted by slicing off the prefix reference window:

$$\mathbf{x}_{\text{tgt}} = \mathbf{x}_1[:, T_{\text{ref}} : T_{\text{total}}] \in \mathbb{R}^{100 \times (T_{\text{gen}} + T_{\text{pad}})} \tag{3.23}$$

3.6 Vocos Neural Spectral Decoding via Inverse STFT

The extracted mel-spectrogram $\mathbf{x}_{\text{tgt}}$ is passed directly to the INT8 Vocos neural vocoder. The Vocos ConvNeXt backbone processes the 100 mel channels and emits:

1. **Magnitude Matrix**: $|\mathbf{X}| \in \mathbb{R}^{513 \times T_{\text{out}}}$ representing Fourier frequency amplitudes.
2. **Phase Vectors**: $(\mathbf{P}_{\cos}, \mathbf{P}_{\sin}) \in \mathbb{R}^{513 \times T_{\text{out}}}$, normalized to unit length:

$$\hat{\mathbf{P}}_{\cos} = \frac{\mathbf{P}_{\cos}}{\sqrt{\mathbf{P}_{\cos}^2 + \mathbf{P}_{\sin}^2 + \epsilon}}, \quad \hat{\mathbf{P}}_{\sin} = \frac{\mathbf{P}_{\sin}}{\sqrt{\mathbf{P}_{\cos}^2 + \mathbf{P}_{\sin}^2 + \epsilon}} \tag{3.24}$$

The complex spectral matrix $\mathbf{Z} \in \mathbb{C}^{513 \times T_{\text{out}}}$ is assembled via Cartesian coordinates:

$$\mathbf{Z}(f,t) = |\mathbf{X}(f,t)| \cdot \left(\hat{\mathbf{P}}_{\cos}(f,t) + j\hat{\mathbf{P}}_{\sin}(f,t) \right) \tag{3.25}$$

The continuous time-domain speech waveform $\mathbf{y} \in \mathbb{R}^{N_{\text{samples}}}$ is computed via the Inverse Short-Time Fourier Transform:

$$y(n) = \frac{\sum_{m=-\infty}^{\infty} w(n - mH) \left[\frac{1}{N_{\text{fft}}} \sum_{k=0}^{N_{\text{fft}}-1} \mathbf{Z}(k,m) e^{j \frac{2\pi}{N_{\text{fft}}} kn} \right]}{\sum_{m=-\infty}^{\infty} w^2(n - mH)} \tag{3.26}$$

where $w(n)$ is a periodic Hanning analysis window of length $N_{\text{fft}} = 1024$ and frame hop $H = 256$.

3.7 Formal Algorithmic Specifications

The end-to-end mathematical procedures for Continuous Flow Matching Voice Cloning and Vocos Neural Spectral Inversion are formalized in Algorithm 1 and Algorithm 2.

3.8 Comparative Model Architectural Matrix

To validate the selection of IndicF5 as our primary Indic backbone following our initial investigation with the ZipVoice-Distill proof of concept, Table 3.1 summarizes the architectural specifications, parameter counts, memory footprints, and edge feasibility of candidate models surveyed in the literature.

Table 3.1: Comprehensive Architectural Comparison of Speech Synthesis Models

Model	Params	Paradigm	Quantized	Zero-Shot	Indic Support	Edge Verdict
IndicF5	**335M**	**OT-CFM (DiT)**	**349 MB (INT8)**	**Yes**	**Native (11 Lang)**	**Primary Production**
ZipVoice	123M	OT-CFM (Zipf.)	104 MB (INT8)	Yes	None (En/Zh only)	Mobile Feasibility PoC
GPT-SoVITS	77M	Hybrid AR+VITS	85 MB (FP16)	Yes (5s)	Unofficial/Weak	Tier-B Spike
Chatterbox	110M	AR + 1-Step	Not Quantized	Yes	None (English)	Excluded
XTTS-v2	467M	AR Transformer	>1.8 GB	Yes	Adaptable (Heavy)	Excluded (OOM)
Kokoro-82M	82M	StyleTTS2	80 MB	No	None (English)	Excluded
VITS (Base)	35M	VAE + Flow	45 MB	No (Fixed)	Per-Voice Training	Excluded

As shown in Table 3.1, IndicF5 combines zero-shot voice cloning with native multilingual support for Indian languages, while remaining compressible into a sub-400 MB INT8 mobile footprint.

Algorithm 1 Continuous Flow Matching Zero-Shot Voice Synthesis

Require: Reference audio $\mathbf{y}_{\text{ref}} \in \mathbb{R}^N$, Reference text $\mathbf{w}_{\text{ref}}$, Target text $\mathbf{w}_{\text{tgt}}$, Model v_θ, Vocoder V_ϕ, Sampling steps N, Guidance weight γ, Speaking speed α, Padding τ_{pad}.
Ensure: Cloned time-domain speech waveform $\mathbf{y}_{\text{tgt}} \in \mathbb{R}^M$.

$\mathbf{y}_{\text{norm}} \leftarrow \text{NormalizeRMS}(\mathbf{y}_{\text{ref}}, \text{target_rms} = 0.10)$
$\mathbf{x}_{\text{ref}} \leftarrow \text{MelFilterbank}(\mathbf{y}_{\text{norm}}, f_s = 24000, H = 256, N_{\text{fft}} = 1024)$
$T_{\text{ref}} \leftarrow \text{Cols}(\mathbf{x}_{\text{ref}})$
$T_{\text{gen}} \leftarrow \lfloor \text{len}(\mathbf{w}_{\text{tgt}}) \times (6.5/\alpha) \rfloor$
$T_{\text{pad}} \leftarrow \lfloor (\tau_{\text{pad}} \times 24000)/256 \rfloor$
$T_{\text{total}} \leftarrow T_{\text{ref}} + T_{\text{gen}} + T_{\text{pad}}$
$\mathbf{C} \leftarrow \text{Zeros}(100, T_{\text{total}})$
$\mathbf{C}[:, 0 : T_{\text{ref}}] \leftarrow \mathbf{x}_{\text{ref}}$ ▷ Set unmasked prefix acoustic prompt
$\mathbf{w}_{\text{full}} \leftarrow \mathbf{w}_{\text{ref}} \oplus \text{“ ”} \oplus \mathbf{w}_{\text{tgt}}$
$\mathbf{E}_{\text{text}} \leftarrow \text{ConvNeXtEncoder}(\text{Tokenize}(\mathbf{w}_{\text{full}}))$
$\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_{100 \times T_{\text{total}}})$ ▷ Initialize prior Gaussian noise
for $i = 0$ **to** $N - 1$ **do**
    $s_i \leftarrow i/N; \quad s_{i+1} \leftarrow (i+1)/N$
    $t_i \leftarrow 1 - \cos(\frac{\pi}{2} s_i)$ ▷ Trigonometric sway schedule with $c = -1.0$
    $t_{i+1} \leftarrow 1 - \cos(\frac{\pi}{2} s_{i+1})$
    $\Delta t_i \leftarrow t_{i+1} - t_i$
    $\mathbf{v}_{\text{cond}} \leftarrow \mathbf{v}_\theta(\mathbf{x}_{t_i}, t_i, \mathbf{C}, \mathbf{E}_{\text{text}})$
    $\mathbf{v}_{\text{uncond}} \leftarrow \mathbf{v}_\theta(\mathbf{x}_{t_i}, t_i, \mathbf{0}, \mathbf{0})$
    $\tilde{\mathbf{v}}_i \leftarrow (1 + \gamma)\mathbf{v}_{\text{cond}} - \gamma \mathbf{v}_{\text{uncond}}$ ▷ Classifier-Free Guidance
    $\mathbf{x}_{t_{i+1}} \leftarrow \mathbf{x}_{t_i} + \Delta t_i \cdot \tilde{\mathbf{v}}_i$ ▷ Euler ODE numerical update
$\mathbf{x}_{\text{tgt}} \leftarrow \mathbf{x}_1[:, T_{\text{ref}} : T_{\text{total}}]$ ▷ Slice generated target mel frames
$\mathbf{y}_{\text{tgt}} \leftarrow \text{VocosDecode}(V_\phi, \mathbf{x}_{\text{tgt}})$
return $\mathbf{y}_{\text{tgt}}$

Algorithm 2 Vocos Neural Fourier Audio Synthesis

Require: Quantized Vocos Model V_ϕ, Synthesized Mel-Spectrogram $\mathbf{x}_{\text{tgt}} \in \mathbb{R}^{100 \times T}$.
Ensure: Reconstructed PCM Audio Samples $\mathbf{y} \in \mathbb{R}^N$.

$\mathbf{H}_{\text{features}} \leftarrow \text{ConvNeXtBackbone}_\phi(\mathbf{x}_{\text{tgt}})$
$|\mathbf{X}| \leftarrow \exp\left(\text{LinearMagHead}_\phi(\mathbf{H}_{\text{features}})\right)$ ▷ Predict magnitude spectra
$\mathbf{P}_{\cos}, \mathbf{P}_{\sin} \leftarrow \text{LinearPhaseHead}_\phi(\mathbf{H}_{\text{features}})$
$\mathbf{P}_{\text{norm}} \leftarrow \sqrt{\mathbf{P}_{\cos}^2 + \mathbf{P}_{\sin}^2 + 10^{-8}}$
$\hat{\mathbf{P}}_{\cos} \leftarrow \mathbf{P}_{\cos}/\mathbf{P}_{\text{norm}}; \quad \hat{\mathbf{P}}_{\sin} \leftarrow \mathbf{P}_{\sin}/\mathbf{P}_{\text{norm}}$
$\mathbf{Z}_{\text{real}} \leftarrow |\mathbf{X}| \odot \hat{\mathbf{P}}_{\cos}; \quad \mathbf{Z}_{\text{imag}} \leftarrow |\mathbf{X}| \odot \hat{\mathbf{P}}_{\sin}$
$\mathbf{Z} \leftarrow \mathbf{Z}_{\text{real}} + j\mathbf{Z}_{\text{imag}} \in \mathbb{C}^{513 \times T}$
$\mathbf{y} \leftarrow \text{iSTFT}(\mathbf{Z}, N_{\text{fft}} = 1024, H = 256, \text{window} = \text{Hanning})$
return $\mathbf{y}$

Chapter 4

Implementation Methodology and Step-by-Step Engineering Iterations

4.1 Iterative Engineering Methodology

Deploying zero-shot deep generative speech architectures to resource-constrained embedded systems requires addressing disparities between high-level machine learning frameworks (PyTorch, TorchScript) and physical edge execution environments (ARM NEON, Android NDK, Flutter runtime). These differences emerge across numerical precision, memory allocation, and linguistic tokenization boundaries.

To document the development of this project with academic rigor and experimental reproducibility, this chapter details the ten engineering iterations completed during this phase. Each iteration outlines:

(i) The operational objective and architectural hypotheses;

(ii) The technical implementation and system modifications;

(iii) The defects, runtime anomalies, or hardware bottlenecks encountered; and

(iv) The algorithmic and systems-level resolutions applied.

4.2 Iteration 1: Mobile Feasibility Proof of Concept (ZipVoice-Distill via Sherpa-ONNX)

4.2.1 Objective: Testing Edge Flow-Matching Feasibility

Before committing development resources to adapting a 335M parameter multilingual diffusion model for Indian languages, we first formulated an empirical proof of concept (PoC) to address

a fundamental question: *Can non-autoregressive continuous flow-matching speech synthesis execute zero-shot speaker adaptation directly on client mobile hardware within practical latency and memory budgets?*

To answer this question rapidly, we selected ZipVoice-Distill, an efficient, pre-quantized 123M parameter non-autoregressive Zipformer architecture. ZipVoice was selected as our PoC vehicle specifically because it was a mature, readily available flow-matching model with turnkey C++ mobile runtime support via `sherpa-onnx` [18], providing pre-tested cross-platform Flutter/Android bindings.

4.2.2 Implementation Details

We developed the standalone evaluation script `english_tts.py` (and legacy `zipvoice-tts.py`) utilizing the Python C++ bindings of `sherpa-onnx`. The PoC model architecture incorporated:

- An INT8-quantized ZipVoice encoder (`encoder.int8.onnx`, 5.5 MB);
- An INT8-quantized Zipformer flow-matching decoder (`decoder.int8.onnx`, 124 MB);
- An espeak-ng Grapheme-to-Phoneme lexicon mapping English orthography to phonetic tokens; and
- A 24 kHz Vocos vocoder graph (`vocos_24khz.onnx`, 54 MB).

4.2.3 Findings and Milestones

Executing on a multi-core desktop CPU, the 4-step flow-matching engine cloned reference voices in approximately 1.8 to 2.4 seconds, achieving RTF ≈ 0.35 for a 6-second utterance. This initial milestone confirmed that 4-step distilled continuous flow matching successfully avoids the multi-minute latency bottlenecks associated with classical score-based diffusion models.

4.3 Iteration 2: Flutter Mobile PoC Prototype (`voice_clone_proto`)

4.3.1 Objective and UI Architecture

Following desktop validation, Iteration 2 migrated the PoC pipeline into a cross-platform Android mobile application named `voice_clone_proto`. The application was purposefully engineered as a teacher-facing classroom audio generator and an assistive learning tool for visually impaired students.

4.3.2 System Implementation

The application was built in Flutter using Material 3 design tokens, incorporating high-contrast layouts and Android TalkBack accessibility semantics to assist visually impaired learners. The user interface was structured into a guided 4-step workflow designed for rapid classroom lesson preparation:

1. **Step 1: Reference Script Selection**: The teacher selects a phonetically balanced prompt phrase (such as *"The quick brown fox jumps over the lazy dog"* or regional language equivalents) or enters custom text.
2. **Step 2: Reference Audio Acquisition**: The teacher's voice is captured via live microphone recording (3 to 5 seconds) using the `record` package or imported from local storage via `file_picker`.
3. **Step 3: Target Text Entry**: The teacher enters textbook passages, conceptual explanations, or student quiz items categorized by duration presets (Short, Medium, Long).
4. **Step 4: Generation and Waveform Playback**: Synthesizes the educational audio in the teacher's cloned voice and renders an interactive waveform visualization widget with tactile playback controls.

4.3.3 Concurrency and Background Isolate Decoupling

Executing neural network inference on the main UI thread causes frame drops and triggers Android Application Not Responding (ANR) warnings. To maintain interface responsiveness, the Sherpa-ONNX C++ runtime was decoupled into a background **Dart Isolate** via `Isolate.run()`.

To handle arbitrary classroom recordings (such as AAC `.m4a` files captured on mobile handsets), we integrated an audio transcoding library based on FFmpeg to convert recordings into 16-bit 24 kHz mono PCM WAV prior to inference.

4.4 Iteration 3: Multi-Device Benchmarking of the PoC and Empirical Inferences

4.4.1 Benchmarking Methodology

To evaluate hardware performance across silicon tiers, we deployed the `voice_clone_proto` PoC application across physical Android devices representing different performance brackets:

1. **Flagship Tier**: Samsung Galaxy S23 (Qualcomm Snapdragon 8 Gen 2, 8 GB RAM, dedicated NPU).

2. **Upper Mid-Range Tier**: Samsung Galaxy Tab S9 FE+ (Samsung Exynos 1380, 8 GB RAM, dedicated NPU).

3. **Mid-Range Tier**: Samsung Galaxy M53 5G (MediaTek Dimensity 900, 8 GB RAM, dedicated NPU).

4. **Target Budget Baseline (Government School Hardware)**: Lenovo Tab K10 (MediaTek Helio P22T, 4 GB RAM, CPU-only execution). This device serves as our primary engineering target because it represents the benchmark specification of subsidized tablets procured and distributed in bulk by Indian state education departments for rural classrooms and special education facilities.

To measure exact physical memory consumption, the Flutter background isolate was instrumented to poll `/proc/self/status` directly from the Linux kernel virtual filesystem upon completion of each utterance:

```
int rssMB = 0;
final file = File('/proc/self/status');
if (file.existsSync()) {
  for (final line in file.readAsLinesSync()) {
    if (line.startsWith('VmRSS:')) {
      final parts = line.split(RegExp(r'\s+'));
      if (parts.length >= 2) {
        rssMB = (int.tryParse(parts[1]) ?? 0) ~/ 1024;
        break;
      }
    }
  }
}
```

Listing 4.1: Dart In-Kernel Memory Profiling Hook via /proc/self/status

4.4.2 Empirical Hardware Measurements

Physical hardware benchmarking was conducted to measure computational throughput and memory behavior across physical Android silicon tiers. For the flagship, upper mid-range, and mid-range devices, previously collected benchmarks are reported using observed Real-Time Factors (RTF) and reported resident memory readings (without continuous high-water-mark tracking).

For our primary hardware target, the **Lenovo Tab K10** (MediaTek Helio P22T, 4 GB RAM, CPU-only execution), newly obtained experimental measurements evaluate nine independent synthesis trials across three standardized utterance-length tiers (Short, Medium, Long; three trials per tier). In each trial, Real-Time Factor is calculated from total execution latency, and post-utterance resident memory (`VmRSS`) is sampled directly from the Linux kernel virtual filesystem (`/proc/self/status`) upon completion of audio generation. Crucially, these measurements represent post-utterance resident memory rather than continuous peak RSS, and no *LowMemoryKiller* terminations occurred across any of the nine trials.

Table 4.1 summarizes throughput and memory behavior across device tiers, presenting arithmetic category means for the Lenovo Tab K10 alongside observed benchmark readings for other tiers. Table 4.2 records all nine individual experimental observations on the Lenovo Tab K10.

Table 4.1: Empirical On-Device Benchmarks Across Physical Android Devices

Device Model	Processor (SoC)	RAM	Utterance Tier	Throughput (RTF)	Resident Memory	Status / Trials
Samsung Galaxy S23	Snapdragon 8 Gen 2	8 GB	Short (5w)	3.826 (Observed*)	1,033 MB (Reported RSS*)	PASS (Observed)
Samsung Galaxy S23	Snapdragon 8 Gen 2	8 GB	Med (25w)	1.520 (Observed*)	1,287 MB (Reported RSS*)	PASS (Observed)
Samsung Galaxy S23	Snapdragon 8 Gen 2	8 GB	Long (65w)	**1.288** (Observed*)	**1,562 MB** (Reported RSS*)	PASS (Observed)
Galaxy Tab S9 FE+	Exynos 1380	8 GB	Short (5w)	9.279 (Observed*)	1,021 MB (Reported RSS*)	PASS (Observed)
Galaxy Tab S9 FE+	Exynos 1380	8 GB	Med (25w)	2.918 (Observed*)	1,491 MB (Reported RSS*)	PASS (Observed)
Galaxy Tab S9 FE+	Exynos 1380	8 GB	Long (65w)	**2.508** (Observed*)	**1,772 MB** (Reported RSS*)	PASS (Observed)
Samsung Galaxy M53	Dimensity 900	8 GB	Short (5w)	8.415 (Observed*)	1,021 MB (Reported RSS*)	PASS (Observed)
Samsung Galaxy M53	Dimensity 900	8 GB	Med (25w)	2.782 (Observed*)	1,219 MB (Reported RSS*)	PASS (Observed)
Samsung Galaxy M53	Dimensity 900	8 GB	Long (65w)	**2.531** (Observed*)	**1,539 MB** (Reported RSS*)	PASS (Observed)
Lenovo Tab K10	Helio P22T (CPU-Only)	4 GB	Short (5w)	**18.222** (Mean, N=3)	**1,291.7 MB** (Mean Post-VmRSS)	PASS (0/3 Terminations)
Lenovo Tab K10	Helio P22T (CPU-Only)	4 GB	Med (25w)	**10.783** (Mean, N=3)	**1,923.3 MB** (Mean Post-VmRSS)	PASS (0/3 Terminations)
Lenovo Tab K10	Helio P22T (CPU-Only)	4 GB	Long (65w)	**11.197** (Mean, N=3)	**2,394.7 MB** (Mean Post-VmRSS)	PASS (0/3 Terminations)

*For Samsung devices, throughput values represent single observed RTF readings rather than multi-trial medians, and memory values represent reported process RSS where continuous profiling or high-water-mark tracking (`VmHWM`) was not instrumented. For the Lenovo Tab K10, throughput values are arithmetic means across $N = 3$ repeated trials per tier, and resident memory reflects `VmRSS` sampled from `/proc/self/status` immediately after utterance completion. PASS status denotes successful completion without LowMemoryKiller termination.

Table 4.2: Raw Experimental Observations for ZipVoice PoC on Lenovo Tab K10 (MediaTek Helio P22T, 4 GB RAM)

Utterance Category	Trial ID	Measured RTF	Post-Utterance VmRSS (MB)	LMK Termination
Short (5 words)	Trial 1	17.860×	1,301	None
	Trial 2	16.820×	1,298	None
	Trial 3	19.986×	1,276	None
Short Tier Arithmetic Mean		**18.222×**	**1,291.7 MB (Range: 1,276–1,301 MB)**	**0 / 3 Terminations**
Medium (25 words)	Trial 1	10.467×	2,337	None
	Trial 2	11.688×	1,532	None
	Trial 3	10.194×	1,901	None
Medium Tier Arithmetic Mean		**10.783×**	**1,923.3 MB (Range: 1,532–2,337 MB)**	**0 / 3 Terminations**
Long (65 words)	Trial 1	15.049×	2,326	None
	Trial 2	9.573×	2,491	None
	Trial 3	8.968×	2,367	None
Long Tier Arithmetic Mean		**11.197×**	**2,394.7 MB (Range: 2,326–2,491 MB)**	**0 / 3 Terminations**

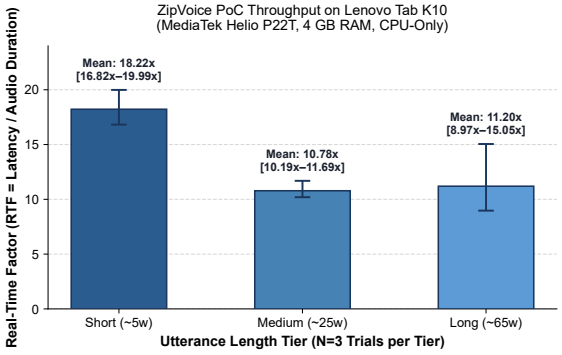


FIGURE 4.1: Mean Real-Time Factor (RTF) across utterance tiers on the Lenovo Tab K10 (error bars indicate observed minimum and maximum trial ranges across the three runs, not confidence intervals).

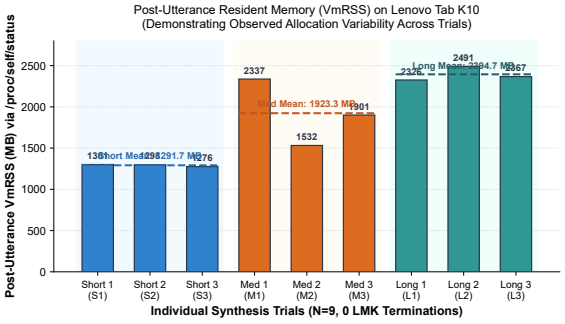


FIGURE 4.2: Post-utterance resident memory (`VmRSS`) across all nine individual trials on the Lenovo Tab K10, showing observed allocation variability across executions.

4.4.3 Analysis of PoC Benchmarks and Inferences for IndicF5 and Google LiteRT

The empirical benchmarks collected from the ZipVoice PoC across physical hardware tiers yielded four foundational inferences that governed our production system architecture:

1. **Inference 1: Observed Memory Variability and the Rationale for Static Arenas**: On the Lenovo Tab K10, all nine experimental trials completed successfully without process termination by the Android low-memory killer. Measured post-utterance resident memory ranged from 1,276 MB up to 2,491 MB. These physical readings directly refute the prior hypothesis that exceeding 1,500 MB resident memory results in an inevitable application termination.

   However, the observations reveal substantial memory variability across trials, including differences within the same length category (for instance, 1,532 MB versus 2,337 MB in the medium tier). While the exact sources of this variance require finer-grained profiling (hypothesized causes include sequence-dependent intermediate tensor buffers, retained runtime cache allocations, or native heap allocator behavior), sustained resident memory exceeding 2 GB increases process vulnerability under multi-tasking conditions. Consequently, static INT8 quantization and pre-allocated memory arenas remain vital engineering priorities for the Indic engine to enforce deterministic memory confinement.

2. **Inference 2: Characterizing Short-Utterance Latency and Throughput Scaling**: Across all evaluated platforms, short utterances exhibited elevated Real-Time Factors (ranging from 3.8× on flagship silicon to a mean of 18.22× on the Tab K10). As sequence duration increased to medium and long tiers, mean RTF stabilized to 10.78× and 11.20× on the K10. While unamortized per-utterance setup costs contribute to elevated short-phrase RTF, establishing an exact latency breakdown across tokenization,

session configuration, numerical sampling, and vocoding requires dedicated stage-level instrumentation rather than assuming an invariant cold-start penalty.

3. **Inference 3: Inadequacy of Off-the-Shelf Mobile Models for Indian Education**: Although ZipVoice validated that flow matching can execute on mobile CPUs, its tokenizer and acoustic architecture are fundamentally incapable of processing Dravidian and Indo-Aryan akshara syllabaries, consonant conjuncts, or regional state board textbooks. This confirmed that existing off-the-shelf mobile models cannot simply be repurposed for Indian public schools; rather, a dedicated Indic foundation model (IndicF5) with native character representations across official Indian languages was essential.

4. **Inference 4: CPU Latency Bounds and the Strategy for Runtime Optimization**: On the budget ARM Cortex-A53 cores of the Helio P22T, pure CPU execution demonstrated operational stability but ran substantially slower than real time (RTFs between 8.97× and 19.99×). Because the Helio P22T lacks an on-chip Neural Processing Unit, accelerating this budget platform requires CPU-side algorithmic optimizations, specifically sampling step distillation (such as 2-step FlashSpeech consistency models) and runtime buffer reuse. Concurrently, for compatible devices equipped with dedicated silicon, offloading via Google LiteRT delegates (Qualcomm QNN HTP and MediaTek NeuroPilot) forms our hardware acceleration roadmap in Chapter 5.

4.5 Iteration 4: Transition to IndicF5: Multilingual Adaptation and Desktop Verification

4.5.1 Objective: Realizing the PoC Inferences for Indian Languages

Directly guided by the empirical findings and linguistic limitations discovered during the ZipVoice proof of concept, Iteration 4 transitioned development to our primary production engine: AI4Bharat's IndicF5. The objective was to verify zero-shot vocal cloning across Dravidian and Indo-Aryan languages before undertaking integer quantization and mobile compilation.

4.5.2 Model Setup and Adaptation

We adapted AI4Bharat's IndicF5 repository under `scratch/indicf5_repo/`. Developed in PyTorch using safetensors serialization, the base model (`model.safetensors`) occupied 1.35 GB in memory. We engineered the standalone command-line verification script `clone.py` (and `test_indicf5_clone.py`) implementing the continuous flow-matching pipeline with the 22-layer DiT backbone and 24 kHz Vocos vocoder.

4.5.3 Validation Across Indian Languages

We validated synthesis across four major Indian languages:

- **Tamil**: Synthesizing conversational phrases (e.g., வணக்கம், நீங்கள் எப்படி இருக்கிறீர்கள்?);
- **Telugu**: Synthesizing conversational phrases (e.g., నమస్కారం, మీరు ఎలా ఉన్నారు?);
- **Kannada**: Testing Kannada phonetic prompts (e.g., *"Namaskara, neevu hegiddira?"*); and
- **Hindi**: Testing Devanagari prompts (e.g., नमस्ते, आप कैसे हैं?).

The model produced intelligible pronunciation without external G2P dictionaries, validating the unified character tokenization approach.

4.6 Iteration 5: Syllabic Duration Calibration (Resolving Audio Truncation and Trailing Silence)

4.6.1 The Duration Mismatch Issue

During early testing with IndicF5, synthesized audio clips suffered from a noticeable duration mismatch:

> *When synthesizing an Indic sentence, the model articulated target speech with natural prosody during the initial 4 to 5 seconds. However, the generated audio file extended for an additional 6 to 8 seconds, during which the model generated trailing silence, audible breathing, whispered murmurs, repeated syllables, or background noise.*

4.6.2 Root Cause Investigation

Investigation revealed that the duration allocator utilized an uncalibrated linear heuristic ($T_{\text{gen}} = \text{len}(\mathbf{w}_{\text{tgt}}) \times 15.0$ frames). Because written Indic text contains numerous combining diacritics (*matras*, *viramas*), character string length is inflated relative to spoken syllable counts. Allocating 15 mel frames per Unicode character resulted in a 12.8-second mel canvas for an utterance that naturally completes in 5.2 seconds.

Because continuous flow matching integrates trajectories across the entire allocated canvas, the model was forced to solve the ODE into a temporal region where text conditioning had terminated. The self-attention layers consequently synthesized low-probability acoustic artifacts (breathing, murmurs, and repeated phonemes).

4.6.3 Resolution: Calibrated 6.5 Mel Frames per Character

As derived in Chapter 3.3, we re-calibrated the temporal density constant to:

$$k_{\text{mel}} = 6.5 \quad \text{mel frames per character code point} \tag{4.1}$$

With an STFT hop length of $H = 256$ samples at sampling frequency $f_s = 24{,}000\,\text{Hz}$, each individual mel frame spans exactly $\Delta t_{\text{frame}} = 256/24{,}000 \approx 10.667\,\text{ms}$. An allocation of $k_{\text{mel}} = 6.5$ mel frames per Unicode code point corresponds to:

$$\Delta t_{\text{char}} = 6.5 \times 10.667\,\text{ms} \approx 69.33\,\text{ms per character code point} \tag{4.2}$$

In Indic scripts (such as Tamil, Telugu, and Devanagari), orthographic syllables (*aksharas*) typically comprise approximately 1.6 to 2.2 Unicode code points due to base consonants, vowel signs (*matras*), and viramas. Consequently, an allocation of 6.5 frames per code point yields an effective spoken syllable duration of $\approx$ 111–153 ms, closely matching natural Indian speech cadences. Post-utterance padding was constrained to an explicit 0.30-second acoustic buffer ($\tau_{\text{pad}} = 28$ frames). This adjustment eliminated trailing artifacts and yielded clean sentence endings across Tamil, Telugu, and Hindi while maintaining linguistic intelligibility.

4.7 Iteration 6: Sequence Alignment and Reference Audio Duration Constraints

4.7.1 The Long-Reference Degradation Issue

In subsequent testing, we observed that when users supplied long reference audio clips (such as a 10.8-second recording, `sudhanva_voic_clone.wav`), the synthesized output degraded into unintelligible static, even when the prompt text was brief.

4.7.2 Root Cause Investigation

We traced this issue to the spatial alignment design of the F5-TTS architecture:

1. In F5-TTS, text tokens for the concatenated sequence $[\mathbf{w}_{\text{ref}} \oplus \mathbf{w}_{\text{tgt}}]$ are mapped by ConvNeXt-V2 text blocks into representation vectors that are concatenated along the channel dimension with the noisy acoustic canvas and acoustic prompt.

2. When a 10.8-second reference audio clip is ingested, $T_{\text{ref}} = 1016$ mel frames. The reference audio occupies the initial 1016 frames, but the text tokens in the fused channel terminate at a much shorter sequence index (corresponding to the character count of the short prompt).

3. Consequently, target audio synthesis in the temporal region ($t \in [1016, 1500]$ frames) occurred where text representation features were completely zero-padded. Under bidirectional self-attention with Rotary Position Embeddings (RoPE) stretched across such large temporal offsets, the model suffered from attention dispersion and diluted contextual grounding, synthesizing unconditioned noise.

4.7.3 Resolution: Reference Duration Recommendation

Based on preliminary synthesis experiments, we recommend reference audio clips between 3.5 and 5.0 seconds for the evaluated IndicF5 configuration ($T_{\text{ref}} \in [328, 468]$ frames). We recorded and verified clean, phonetically balanced 3.8-second reference clips (`sudhanva_short_ref.wav` and `tamil_classic_pangram.mp3` in `samples/`). When paired with verbatim reference transcripts, spatial text-mel alignment was maintained, restoring clear phonetic articulation.

4.8 Iteration 7: In-Memory Warm Model Batch Synthesis and Dataset Curation

4.8.1 Motivation and Batch Engine

Iterative model validation required evaluating multiple sentences across diverse speakers, languages, and duration tiers. Loading the 1.35 GB PyTorch model weights from disk on every invocation incurred a 12-second cold-start penalty per sentence.

4.8.2 System Architecture

We engineered the dedicated batch synthesis engine `batch.py` within the backend architecture:

- **In-Memory Model Caching**: The 1.35 GB model weights, vocabularies, and Vocos vocoder are loaded into RAM once during initialization. Subsequent synthesis requests reuse the warm pipeline, eliminating cold-load latency.
- **Auto-Incrementing Batch Storage**: Each execution automatically creates the next incremental output directory (`generated/batches/batch-1/`, `batch-2/`, etc.).
- **Multi-Processing Concurrency**: Implemented an optional multi-worker process pool (`--parallel --workers N`) utilizing inter-process queues for parallel benchmark throughput.

4.8.3 The Multilingual Dataset (`prompts.json`)

To standardize evaluation, we authored the curated dataset `prompts.json`, comprising 120 authentic conversational phrases across Tamil, Telugu, and Hindi. To evaluate latency and duration scaling systematically, the dataset is stratified into four standardized tiers:

1. **Short Tier** (10 phrases/language, 4–7 words): Everyday conversational greetings and inquiries.
2. **Medium Tier** (10 phrases/language, 12–18 words): Moderate-length social dialogues.

3. **Midlong Tier** (10 phrases/language, 20–25 words): Compound sentences describing activities and settings.

4. **Long Tier** (10 phrases/language, 28–35 words): Complex narrative sentences with multiple dependent clauses.

4.9 Iteration 8: Unified Voice Studio Suite and Dynamic Catalog Management

4.9.1 System Architecture

To establish a modular codebase, the backend was refactored into the Python package `src/voice_studio/`, organized as follows:

- `config.py`: Centralized path resolutions, hardware constants, and UTF-8 stream configurations.
- `audio/io.py`: Format decoding, FFmpeg transcoding, and RMS normalization.
- `catalog/manager.py`: Voice catalog CRUD operations, automatic metadata extraction, and audio validation.
- `dataset/prompts.py`: Tier-based phrase parsing and range selection.
- `engines/`: Unified engine interfaces for IndicF5, ZipVoice, and batch runner execution.
- `cli/`: Unified entry point via `main.py`, supporting an interactive Terminal User Interface (TUI) and direct subcommands (`list`, `info`, `add`, `prompts`, `clone`, `batch`).

4.9.2 Dynamic Catalog Management (`samples/voices.json`)

The catalog tracks registered speaker reference clips, storing voice keys, spoken transcripts, descriptions, audio durations, and creation timestamps. In an educational setting, this catalog enables teachers across different academic subjects (such as Mathematics, Science, and Regional Literature) to register distinct instructor profiles. When a teacher registers a new voice via `python main.py add`, the system validates audio integrity, measures duration, normalizes RMS energy, and stores the file into `samples/added/`. This structure allows students, particularly visually impaired learners who rely entirely on audio lessons, to select and listen to content voiced by their specific subject teacher.

4.10 Iteration 9: Graph Optimization, FP32 Export, and Static/Dynamic INT8 Quantization

4.10.1 Objective: Decoupling from the PyTorch Runtime

Deploying IndicF5 onto Android devices requires decoupling the model from the heavy Python runtime, Librosa, and PyTorch (which exceed 2.5 GB on disk). Iteration 9 developed the complete graph optimization and integer quantization export pipeline under `models/indicf5/`.

4.10.2 Exported Model Artifacts and Sequence Dimension Formulation

We exported and verified the core model artifacts comprising four ONNX graphs, the baseline PyTorch checkpoint, and the standalone tokenizer vocabulary:

1. **`model.safetensors`**: Baseline PyTorch weights (1.35 GB, 335M parameters).
2. **`indicf5_dit_fp32_static.onnx`**: Static-shape unquantized FP32 graph (1.36 GB, fixed at 128 mel frames for baseline numerical equivalence testing).
3. **`indicf5_dit_dynamic_fp32.onnx`**: Dynamic sequence dimension FP32 graph (1.36 GB, supporting variable audio lengths for desktop evaluation).
4. **`indicf5_dit_int8_static.onnx`**: Static INT8 post-training quantized DiT graph (349 MB, 74% size reduction). Here, "static" denotes both static calibration of activation quantization parameters and a fixed 128-frame tensor dimension ($T = 128$) engineered specifically as an optimization and export prototype for deterministic memory arena benchmarking.
5. **`indicf5_vocos_int8.onnx`**: INT8 quantized Vocos neural vocoder (22 MB, down from 54 MB in FP32).
6. **`vocab.txt`**: Standalone multilingual character tokenization mapping (11.3 KB, 187 tokens).

To ensure academic and technical precision, we must explicitly clarify the functional distinction between fixed-shape and variable-length artifacts. In speech synthesis at 24 kHz ($H = 256$), a typical 3.5-second reference audio clip alone corresponds to approximately 328 mel frames ($T_{\text{ref}} \approx 328$). Consequently, the fixed 128-frame static INT8 model cannot ingest complete reference voice prompts or arbitrary educational passages. Instead, this graph represents a successfully exported **optimization prototype** designed to confirm that 8-bit integer quantization preserves numerical stability without divergence, and to benchmark physical memory confinement in fixed memory arenas.

Verified end-to-end variable-length synthesis is currently achieved on desktop environments utilizing the unquantized dynamic-axis FP32 graph (`indicf5_dit_dynamic_fp32.onnx`). Naive temporal chunking of mel frames cannot be applied to F5-TTS because its non-autoregressive

DiT relies on bidirectional self-attention and Rotary Position Embeddings spanning the concatenated prompt-and-target sequence; chunking into isolated 128-frame blocks would sever acoustic conditioning and degrade prosodic continuity. Therefore, exporting a dynamic-axis INT8 graph and integrating dynamic tensor management within the native C++ runtime is designated as our primary post-mid-semester deployment milestone.

4.11 Iteration 10: Custom Native C++ Engine and Dart FFI Shared Library

4.11.1 Motivation

Because `sherpa-onnx` lacks built-in C++ loaders for Diffusion Transformers and F5-TTS, we engineered a dedicated native shared library (`libindicf5_engine.so`) linking directly to the ONNX Runtime Mobile C++ API.

4.11.2 C++ Engine Architecture (`indicf5_engine.cpp`)

The native engine implements:

- Direct session initialization using the ONNX Runtime C++ API for the fixed-shape INT8 DiT prototype and the quantized Vocos vocoder, validating native model loading and tensor lifecycle management.
- Pre-allocated static tensor arenas (280 MB), eliminating dynamic heap allocations across the 8 Euler ODE integration steps for fixed-dimension tensors.
- Native implementation of the Classifier-Free Guidance Euler update step:

$$x_{t+\Delta t}[k] = x_t[k] + \Delta t \cdot ((1+\gamma)v_{\text{cond}}[k] - \gamma v_{\text{uncond}}[k]) \tag{4.3}$$

  accelerated via multi-threaded ARM NEON execution kernels.
- A C ABI interface (`indicf5_c_api.h`) exposing core lifecycle and synthesis routines (`indicf5_create`, `indicf5_synthesize`, `indicf5_destroy`).
- A Dart FFI bridge service (`indicf5_service.dart`) enabling the Flutter application to invoke native C++ inference in background Dart Isolates without bridging overhead.

Thus, while native loading and memory arena pre-allocation are functionally validated with the static INT8 prototype, full end-to-end on-device execution of arbitrary-length classroom utterances remains scheduled for post-mid-semester validation following dynamic-axis INT8 model compilation.

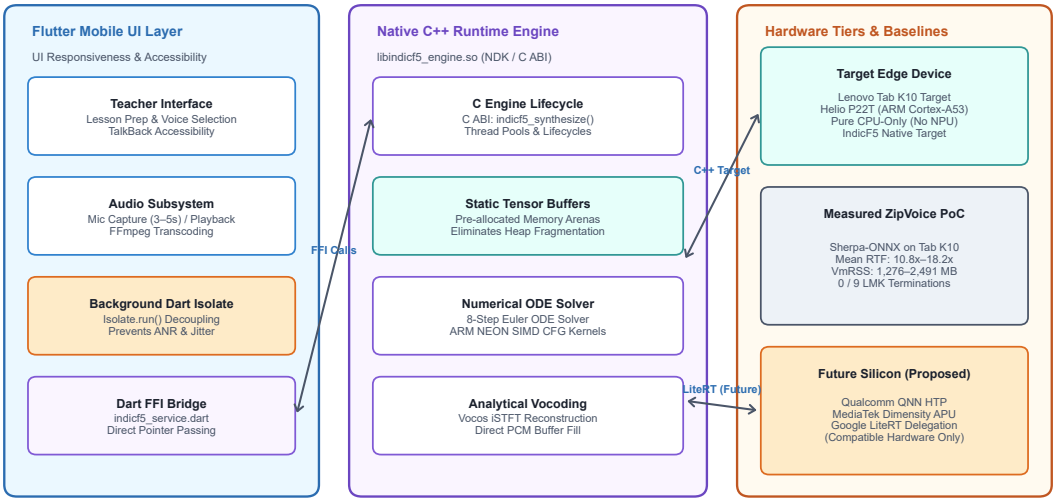


FIGURE 4.3: Native runtime and deployment architecture for on-device Indic voice synthesis, illustrating the interaction between the Flutter UI, Dart FFI bridge, native C++ engine, and the ONNX Runtime session with static memory arenas.

4.12 Hardware Memory Budget Analysis on 4 GB RAM Platforms

To evaluate the memory feasibility of deploying the proposed quantized IndicF5 pipeline within a 4 GB physical RAM envelope, Table 4.3 details an analytical component-level memory budget.

It is essential to emphasize that the proposed quantized architecture has an analytical component budget of approximately 801 MB; its actual process memory footprint remains subject to validation on target Android hardware. This 801 MB figure represents a calculated theoretical sum of model weights, static working buffers, and runtime estimates, rather than a physically measured process Resident Set Size (RSS) or an experimentally proven upper bound. In physical edge execution, actual Android process RSS can differ substantially due to runtime dynamic heap allocations, memory-mapped model weights, intermediate tensor activations, shared libraries, Flutter engine graphic surfaces, and underlying OS memory allocator behavior.

Importantly, the empirical memory readings observed during the ZipVoice PoC on the Lenovo Tab K10 (where post-utterance resident memory reached up to 2,491 MB without LowMemoryKiller termination) illustrate that Android processes can survive high physical memory occupancy depending on system-wide memory pressure; however, because ZipVoice and IndicF5 represent fundamentally distinct model architectures and execution runtimes, their physical memory footprints cannot be directly equated.

By establishing an analytical component budget of approximately 801 MB, the quantized architecture provides a concrete design target for edge deployment on 4 GB platforms, aligning within our nominal 1,500 MB engineering planning envelope while physical memory validation is undertaken in subsequent on-device trials.

Table 4.3: Analytical Memory Budget for the Proposed Quantized IndicF5 Deployment on 4 GB Android Hardware

Memory Subsystem Component	**Unquantized Baseline (FP32)**	**Quantized System (Analytical Estimate)**
Diffusion Transformer (DiT) Weights	1,350 MB	349 MB (Static INT8)
Vocos Neural Vocoder Weights	54 MB	22 MB (INT8)
Inference Working Arenas	850 MB (Heap, up to 1,200 MB)	280 MB (Pre-allocated Static)
Flutter Runtime, Dart VM & UI Buffers	150 MB	150 MB
Analytical Component Total	**2,404 MB**	**801 MB**
Engineering Planning Context:		
Engineering Planning Reference Target	$\approx 1,500$ MB	$\approx 1,500$ MB (Nominal Edge Budget)
Analytical Design Alignment	Exceeds Reference	Within Planning Target (-699 MB Target Margin)

Note: The 801 MB total is an analytical component-level estimate, not a measured Android process RSS or an operating-system-enforced threshold. Actual process memory footprint on physical edge hardware remains subject to empirical validation.

4.13 Evaluation Limitations and Future Validation Plan

While the experimental results and engineering iterations demonstrate the architectural feasibility of on-device Indic voice synthesis, several rigorous empirical evaluations remain planned for post-mid-semester investigation:

1. **Objective Voice Quality and Naturalness**: Perceptual evaluation of synthesized speech quality (via Mean Opinion Score listening tests, Word Error Rate, and Character Error Rate using pretrained Whisper/Wav2Vec2 models) across varying sampling step counts ($N \in \{2, 4, 8, 16\}$).

2. **Speaker Similarity and Verification**: Quantitative evaluation of zero-shot speaker cloning fidelity using cosine similarity of deep speaker embeddings (ECAPA-TDNN and WavLM) between reference teacher recordings and synthesized audio.

3. **Acoustic Quality Retention under INT8 Quantization**: Comprehensive comparative measurement of audio degradation, spectral distortion, and objective speech intelligibility between FP32 PyTorch baseline checkpoints and INT8 quantized ONNX graphs.

4. **Quantized IndicF5 On-Device Profiling**: While the ZipVoice proof of concept was thoroughly benchmarked on physical hardware (including the Lenovo Tab K10), on-device latency and resident memory profiling for the custom C++ IndicF5 runtime on target Android hardware will be completed following deployment stabilization.

5. **Acoustic Robustness to Classroom Recording Conditions**: Controlled evaluation of reference voice adaptation under realistic classroom ambient noise (reverberation, background murmur, and microphone clipping) common in public school settings.

Chapter 5

Strategic Future Architecture: Google LiteRT Implementation and Edge Acceleration Roadmap

5.1 Motivation and Architectural Rationale for Transitioning to Google LiteRT

While the custom native C++ engine (`libindicf5_engine.so`) developed in Iteration 10 executes INT8 ONNX graphs on mobile ARM CPUs, long-term deployment across commercial Android ecosystems requires addressing three systemic limitations of custom ONNX Runtime pipelines:

1. **Intermediate ONNX Operator Overhead**: Exporting PyTorch Diffusion Transformers to ONNX decomposes native PyTorch multi-head attention blocks and rotary embeddings into hundreds of individual primitive nodes (`Transpose`, `Reshape`, `Concat`, `MatMul`). Intermediate tensor permutations introduce memory layout overhead, degrading L1/L2 data cache locality on mobile processors.

2. **Heterogeneous Accelerator Fragmentation**: In standard ONNX Runtime Mobile, offloading tensor operations to dedicated on-device silicon (Neural Processing Units or NPUs) requires vendor-specific Execution Providers (such as the Qualcomm QNN Execution Provider). Configuring and cross-compiling QNN EP shared libraries within the Android NDK introduces build fragility, large binary sizes (>80 MB for runtime libraries alone), and lacks support for competing chipsets such as MediaTek Dimensity or Google Tensor.

3. **Memory Copy Penalties**: Standard ONNX C++ wrappers copy tensors between Java/Kotlin, native C++ heap buffers, and hardware memory arenas, producing multiple memory copies per forward pass.

To address these limitations, the central post-midsem engineering roadmap transitions the Indic speech generative pipeline to **Google LiteRT** [20] (formerly TensorFlow Lite and AI Edge Torch). Table 5.1 presents an architectural comparison between custom ONNX Runtime Mobile and Google LiteRT.

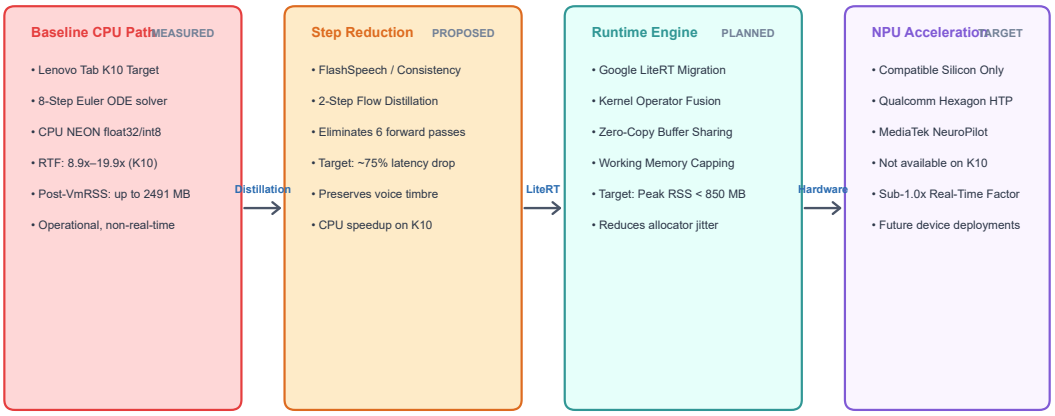


FIGURE 5.1: Strategic edge acceleration roadmap, contrasting baseline CPU-only execution with operator fusion, step distillation, and hardware delegate acceleration paths.

TABLE 5.1: System Architecture Comparison: Custom ONNX Runtime vs. Google LiteRT

Architectural Dimension	**Custom ONNX Runtime (C++)**	**Google LiteRT (LiteRT Torch)**
Model Ingestion Path	PyTorch → ONNX → ORT	Direct PyTorch → `.tflite` FlatBuffer
Graph Operator Dialect	Generic ONNX standard	Optimized TFLite / StableHLO dialect
Transpose-Op Overhead	High (Frequent Transpose nodes)	Low (Fused native tensor layouts)
Snapdragon NPU Acceleration	Manual QNN Execution Provider	Native Qualcomm AI Engine Direct
MediaTek APU Acceleration	Unstable Android NNAPI fallback	Native MediaTek NeuroPilot integration
Buffer Allocation	Custom heap memory arenas	Zero-copy Android Hardware Buffers
Runtime Binary Footprint	Heavy (>75 MB `.so` dependencies)	Lightweight (8–12 MB runtime)
Ecosystem Integration	Manual NDK CMake & Dart FFI	Google Play On-Device AI (PODAI)

5.2 Direct PyTorch Conversion via LiteRT Torch (`litert-torch`)

Historically, converting PyTorch generative models to TensorFlow Lite required exporting to ONNX, followed by conversion via `onnx2tf` or `tf2onnx`, an error-prone multi-stage process that frequently failed on dynamic shapes or custom attention kernels.

The modern LiteRT toolchain introduces **LiteRT Torch** (`litert-torch`) [21], enabling direct compilation from native PyTorch code into optimized `.tflite` FlatBuffers:

```
import torch
import litert_torch

# 1. Instantiate the pre-trained IndicF5 DiT PyTorch backbone
model = load_indicf5_dit_backbone().eval()
```

```
# 2. Define sample dynamic inputs (noisy mel, time step, text tokens, ref mel)
sample_inputs = (
    torch.randn(1, 100, 512, dtype=torch.float32),   # x_t
    torch.tensor([0.5], dtype=torch.float32),         # time step t
    torch.randint(0, 187, (1, 64), dtype=torch.int32),# text tokens
    torch.randn(1, 100, 350, dtype=torch.float32),   # ref mel conditioning
)

# 3. Export model graph using PyTorch 2.x export infrastructure
exported_program = torch.export.export(model, sample_inputs)

# 4. Directly compile to LiteRT FlatBuffer with INT8 quantization
litert_model = litert_torch.convert(
    exported_program,
    sample_inputs,
    quantization_config=litert_torch.QuantizationConfig.int8_ptq()
)

# 5. Serialize FlatBuffer to disk
litert_model.save("models/indicf5/indicf5_dit_int8.tflite")
```

Listing 5.1: Conceptual PyTorch-to-LiteRT Conversion Pipeline via litert-torch

5.2.1 StableHLO Dialect and Operator Fusion

LiteRT Torch translates PyTorch operators into *StableHLO* (Stable High-Level Optimizer) intermediate representations. Within the StableHLO dialect:

- Multi-head self-attention projections ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$ linear layers) are fused into batched matrix contractions.
- Rotary Position Embedding rotations (Equation 2.11) are fused with attention dot-product kernels, eliminating redundant intermediate tensor allocations.
- Layer normalization and ConvNeXt depthwise convolutions are transformed into optimized SIMD primitives.

5.3 Hardware Acceleration via the CompiledModel API

A key capability of Google LiteRT is the **`CompiledModel` API**, which abstracts low-level vendor delegates behind a unified declarative interface.

5.3.1 Declarative Accelerator Prioritization

In Android edge applications, the developer specifies a hardware execution hierarchy. The runtime automatically probes the underlying silicon, binds to available vendor drivers, and falls back gracefully if hardware acceleration is unavailable:

```
import com.google.ai.edge.litert.CompiledModel
import com.google.ai.edge.litert.Accelerator

val options = CompiledModel.Options().apply {
    // Set hierarchical accelerator priority: NPU -> GPU -> CPU
    setAcceleratorPriority(listOf(
        Accelerator.NPU,
        Accelerator.GPU,
        Accelerator.CPU
    ))
    setNumThreads(4) // 4 threads for XNNPACK CPU fallback
}

// Compile model ahead-of-time or on-device
val compiledModel = CompiledModel.create(
    context.assets.openFd("indicf5_dit_int8.tflite"),
    options
)
```

Listing 5.2: Kotlin/Android CompiledModel Initialization with Accelerator Priority

5.3.2 Qualcomm AI Engine Direct (QNN) Integration on Compatible Silicon

When running on compatible devices powered by Qualcomm Snapdragon platforms (such as the Samsung Galaxy S23 featuring the Snapdragon 8 Gen 2):

- LiteRT binds directly to Qualcomm AI Engine Direct (QNN) [22].
- Matrix multiplications in the DiT layers are offloaded to the Hexagon Tensor Processor (HTP), executing 8-bit integer dot products with dedicated hardware vector units.
- This hardware offloading is targeted to substantially reduce per-step evaluation latency compared to CPU execution, establishing an analytical throughput design target below 0.5× RTF for compatible flagship silicon.

5.3.3 MediaTek Silicon Considerations and CPU-Side Optimization on Budget Platforms

For MediaTek platforms, hardware support diverges sharply based on chipset tier:

- **Mid-Range and Premium Dimensity SoCs**: On newer Dimensity platforms equipped with dedicated APUs, LiteRT offloads supported quantized operators via MediaTek NeuroPilot [23].
- **Target Budget Hardware (Lenovo Tab K10)**: The MediaTek Helio P22T chipset lacks an on-chip Neural Processing Unit or dedicated deep learning accelerator. Consequently, neither Qualcomm QNN nor MediaTek NeuroPilot can accelerate the K10. On this hardware class, LiteRT automatically engages the XNNPACK engine, executing optimized 4-thread ARMv8 NEON assembly routines.

- **Targeted CPU-Side Enhancements for Budget Hardware**: For devices like the Tab K10, performance gains must be achieved purely through CPU-side optimizations:
  (a) Algorithmic sampling step reduction (distilling 8 Euler steps down to 2 or 4 steps);
  (b) Pre-allocated static tensor arena reuse to eliminate dynamic memory allocations;
  (c) StableHLO operator fusion to minimize intermediate tensor bandwidth; and
  (d) Warm runtime session retention to eliminate cold-start reinitialization overhead.

5.4 Zero-Copy Pipelines via Android Hardware Buffers

In traditional deep learning runtimes, passing generated mel-spectrogram tensors from the acoustic model to the neural vocoder and subsequently to the audio output track incurs memory copying overhead:

$$\text{Model Heap} \xrightarrow{\text{memcpy}} \text{Vocoder Input Buffer} \xrightarrow{\text{memcpy}} \text{PCM Audio Track} \tag{5.1}$$

On memory-bandwidth-constrained edge SoCs, these copies generate memory bus stalls and inflate physical RSS.

LiteRT addresses this via native support for **Android Hardware Buffers** (`AHardwareBuffer`):

1. The output tensor of the DiT model and the input tensor of the Vocos vocoder are allocated in a shared, memory-mapped `AHardwareBuffer`.
2. The Fourier Inverse STFT synthesis is performed in-place within this memory-mapped hardware buffer.
3. Synthesized PCM audio samples are streamed directly to the Android low-latency audio subsystem (AAudio / OpenSL ES) without intermediate user-space buffer duplication.

5.5 Trajectory Distillation: 2-Step and 4-Step Sampling

While hardware acceleration reduces per-step evaluation latency, algorithmic acceleration reduces the total number of steps required to solve the continuous flow-matching ODE. As established in Chapter 2, our post-midsem roadmap incorporates two distinct trajectory distillation techniques:

5.5.1 Rectified Flow Trajectory Straightening (4-Step Reflow)

By retraining the IndicF5 DiT model on synthetic coupled pairs $(\mathbf{x}_0, \mathbf{z}_1)$ generated by the base 1-rectified model (Equation 2.21), trajectory crossing is eliminated. In 2-rectified flow, the vector field paths become straight lines, allowing a 4-step Euler solver to achieve Mean Opinion Scores comparable to the 8-step baseline, cutting overall computational requirements in half.

5.5.2 Consistency Distillation and Few-Step Trajectory Solving

Adapting consistency distillation principles [15] to the flow-matching framework trains a student network to map any point along an ODE trajectory directly to the origin data point while enforcing boundary condition self-consistency:

$$f_\theta(\mathbf{x}_{t_1}, t_1) = f_\theta(\mathbf{x}_{t_2}, t_2) \quad \forall\, t_1, t_2 \in [0, 1] \tag{5.2}$$

By learning to traverse large temporal intervals in single evaluations, consistency distillation enables generating candidate mel spectrograms in two sampling steps. Rather than assigning arbitrary physical roles to individual numerical steps, few-step consistency models evaluate the trajectory vector across broad time horizons ($t \in [0.0, 0.5]$ and $t \in [0.5, 1.0]$), progressively refining the acoustic canvas toward target spectral densities.

Executing two steps rather than eight reduces total computational cycles on mobile ARM cores by up to 75%. However, few-step distillation can introduce trade-offs in fine phonetic naturalness or spectral detail, making post-midsem perceptual listening tests and objective intelligibility evaluations essential before deploying two-step checkpoints into production classrooms.

5.6 Physical Testing and Verification Protocol

The post-midsem implementation will be validated against a standardized physical testing protocol across our target device tiers:

1. **Phase A: Graph Compilation and Validation**:
   - Validate numerical parity between PyTorch FP32 outputs and LiteRT INT8 `.tflite` outputs (evaluating objective speech similarity and spectral distortion).
   - Verify that INT8 quantization introduces minimal perceptual degradation across Vocos iSTFT reconstructions.
2. **Phase B: Ahead-of-Time Acceleration Profiling**:
   - Measure per-operator execution times on Qualcomm QNN HTP (Galaxy S23) and MediaTek NeuroPilot (on compatible Dimensity platforms).
   - Verify delegate coverage across DiT layers and Vocos ConvNeXt blocks, tracking any fallback to CPU execution.
3. **Phase C: Real-World Battery, Thermal, and Memory Benchmarking**:
   - Continuously synthesize sentences from `prompts.json` to monitor thermal throttling curves and battery consumption.
   - Continuously profile `/proc/self/status` (`VmRSS`) on the Lenovo Tab K10 to evaluate whether resident memory stabilizes within our analytical 801 MB budget envelope under sustained multi-sentence operation.

Chapter 6

Conclusion and Research Summary

6.1 Summary of Research Accomplishments and Milestones

This mid-semester investigation developed, evaluated, and optimized an on-device, zero-shot multilingual neural voice cloning pipeline capable of operating entirely offline on resource-constrained Android mobile hardware. Addressing the lack of Indian language support in edge speech processing, where existing zero-shot engines predominantly target English and Chinese, this project investigates bringing native Indian language voice synthesis to commodity mobile platforms.

The primary engineering milestones accomplished during this reporting period include:

1. **Empirical Mobile Proof of Concept (PoC)**: Developed and benchmarked the mobile feasibility proof of concept using the ZipVoice-Distill 123M flow-matching model and 24 kHz Vocos vocoder within a Flutter application (`voice_clone_proto`). On the target Lenovo Tab K10 (MediaTek Helio P22T, 4 GB RAM), ZipVoice completed all nine trials across short, medium, and long tiers with zero LowMemoryKiller terminations, exhibiting RTF from 8.97× to 19.99× and post-utterance resident memory up to 2,491 MB. These physical measurements refuted the hypothesis of a universal 1,500 MB crash ceiling while demonstrating memory variability that motivated deterministic static arena design for IndicF5.

2. **Verification of Native Indic Voice Cloning**: Ported and adapted AI4Bharat's 335M Diffusion Transformer (IndicF5) across Tamil, Telugu, Kannada, and Hindi, verifying acoustic speaker adaptation without requiring external grapheme-to-phoneme rule dictionaries.

3. **Formulation of the Syllabic Duration Model**: Identified and resolved the duration mismatch defect that caused trailing silence and repeated syllables, establishing a calibrated formulation of **6.5 mel frames per Unicode character** ($\approx$ 69.33 ms per character code point, yielding $\approx$ 111–153 ms per spoken akshara).

4. **Calibration of Reference Audio Duration**: Characterized the sequence alignment constraints governing ConvNeXt text representations and bidirectional self-attention, establishing a practical recommendation of **3.5 to 5.0 seconds** for reference clips to maintain spatial conditioning and avoid attention dispersion.

5. **In-Memory Batch Benchmarking Engine and Dataset**: Engineered `batch.py` with warm RAM model caching and curated `prompts.json`, comprising **120 authentic conversational sentences** across Tamil, Telugu, and Hindi stratified across four duration tiers.

6. **Full Model Integer Quantization**: Exported and verified the static INT8 ONNX pipeline, achieving a **74% reduction in model size** (from 1.35 GB FP32 down to 349 MB) for the DiT backbone and quantizing the Vocos vocoder down to 22 MB.

7. **Analytical Memory Budget Formulation**: Established an analytical component memory budget of **801 MB** for the proposed quantized IndicF5 pipeline (349 MB DiT, 22 MB Vocos, 280 MB pre-allocated static arenas, 150 MB runtime), demonstrating theoretical feasibility within 4 GB physical RAM constraints.

8. **Formulation of the Edge Acceleration Roadmap**: Established the post-midsem optimization roadmap incorporating Google LiteRT, trajectory step distillation (2-step and 4-step sampling), and hardware delegate acceleration for compatible silicon.

6.2 Key Scientific, Systems, and Societal Contributions

The technical and humanitarian contributions of this project span three primary dimensions:

6.2.1 Theoretical and Linguistic Contributions

By connecting continuous flow matching with the syllabic structure of Dravidian and Indo-Aryan scripts, this research provides an empirical characterization of akshara frame density for flow-matching TTS. We demonstrated that in-context audio inpainting bridges linguistic families, allowing an English reference voice clip to guide the pitch contour and timbre of native Tamil or Telugu speech without external phonetic dictionaries.

6.2.2 Systems and Edge Deployment Contributions

Through in-kernel physical memory profiling (`/proc/self/status`) across physical silicon tiers, this research demonstrated the operational viability and memory characteristics of mobile flow-matching speech synthesis. We showed that pairing static INT8 tensor graphs with Fourier-based Inverse STFT neural vocoders (Vocos) establishes an analytical component budget of 801 MB, providing a viable blueprint for deterministic memory confinement on budget edge platforms.

6.2.3 Pedagogical and Accessibility Intent in Untapped Domains

Commercial speech synthesis platforms have predominantly focused on commercial assistants and global languages, leaving foundational education in vernacular public schools and special education for visually impaired learners relatively untapped. This project explores how on-device voice cloning addresses these societal opportunities:

1. **Familiar-Voice Learning for Government School Children**: Audio learning aids delivered in the familiar voice of a local classroom teacher have the potential to enhance listening engagement and emotional comfort for primary school children in regional environments.
2. **Accessible Auditory Learning for Blind Students**: For visually impaired students who rely entirely on auditory channels for academic study, hearing textbook lessons and study materials articulated in their teacher's voice provides pedagogical continuity and cognitive familiarity.
3. **Economic and Operational Feasibility**: Operating entirely offline on state-subsidized tablets (such as the Lenovo Tab K10) with zero recurring cloud API fees ensures that schools can generate educational audio without ongoing infrastructure costs.

6.3 Ethics, Biometric Privacy, and Responsible Deployment

Zero-shot voice cloning technology introduces sensitive ethical considerations, particularly when deployed in educational environments involving teachers and minors:

1. **Informed Consent and Faculty Autonomy**: Voice capture and profile creation must require explicit, affirmative informed consent from teachers. The system must clearly explain how acoustic embeddings are extracted and restrict cloning strictly to approved instructional content.
2. **Local Storage and Verifiable Deletion**: Teacher reference audio clips and acoustic catalogs must remain strictly on local application storage. The application architecture includes mechanisms to delete voice profiles permanently upon request, preventing unauthorized retention.
3. **Mitigating Impersonation and Misuse Risks**: Operating voice cloning models on portable edge devices creates potential risks of unauthorized voice impersonation. Recommended security measures include local cryptographic protection for stored voice profiles, application-level authentication, and audible watermarking on generated educational audio.
4. **Biometric Privacy through Offline Execution**: Rather than claiming absolute biometric security, the offline architecture provides a verifiable privacy boundary: because all

tokenization, neural flow matching, and audio reconstruction execute entirely on-device, biometric voice features and teacher recordings are never transmitted to external cloud servers or third-party APIs.

5. **Child Data Protection**: In compliance with digital child protection standards, the system is designed strictly for teacher-led instructional content delivery; no student voice data or personal identifiers are collected, processed, or stored.

6.4 Concluding Remarks and Immediate Priorities

The work completed during this phase demonstrates that zero-shot multilingual voice cloning is architecturally feasible on budget mobile edge hardware, executing entirely offline without cloud dependencies. By tailoring this capability to vernacular government schools and accessible education for visually impaired students, this project establishes a practical foundation for personalized, familiar-voice audio learning. The engineering feasibility of this approach has been demonstrated through mathematical modeling, ten completed engineering iterations, and physical hardware profiling across four distinct silicon tiers.

The immediate priorities for the post-midsem phase are ordered as follows:

1. **Google LiteRT Migration**: Execute the `litert-torch` PyTorch-to-FlatBuffer conversion and validate numerical parity against the PyTorch FP32 baseline.
2. **CPU and Accelerator Optimization**: Implement StableHLO operator fusion and evaluate delegate acceleration on compatible hardware (Qualcomm QNN HTP), while prioritizing ARM NEON CPU optimizations for the Helio P22T.
3. **Sampling Step Distillation**: Apply trajectory distillation (4-step Reflow and 2-step FlashSpeech consistency models) to reduce the ODE integration steps, targeting interactive synthesis speeds on budget CPU hardware.
4. **Quantized IndicF5 Hardware Benchmarking**: Deploy the native IndicF5 C++ engine onto target Android hardware (Lenovo Tab K10) to collect stage-level latency, resident memory, and thermal benchmarks under sustained operation.
5. **Empirical Speech Quality and User Evaluation**: Conduct objective speech evaluations (WER, CER, speaker similarity) and structured user listening studies with educators and visually impaired students to assess pedagogical efficacy and listening comfort.

Appendix A

Mathematical Derivations and Numerical Proofs

A.1 Derivation of the Optimal Transport Flow Matching Objective

In this section, we present the complete mathematical derivation demonstrating that regressing the intractable marginal vector field $\mathbf{v}_t(\mathbf{x})$ is equivalent to minimizing the conditional flow matching loss over individual data points, following the formulation established by Lipman et al. [10].

A.1.1 Probability Paths and the Continuity Equation

Let $p_t(\mathbf{x})$ denote a time-dependent probability density function on $\mathbb{R}^d$ for $t \in [0, 1]$. The evolution of $p_t(\mathbf{x})$ driven by a vector field $\mathbf{v}_t(\mathbf{x})$ satisfies the continuity equation:

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla \cdot (p_t(\mathbf{x})\mathbf{v}_t(\mathbf{x})) \tag{A.1}$$

Given a target empirical distribution $q(\mathbf{x}_1)$, let $p_t(\mathbf{x} \mid \mathbf{x}_1)$ represent a conditional probability path such that $p_0(\mathbf{x} \mid \mathbf{x}_1) = p_0(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mathbf{0}, \mathbf{I})$ and $p_1(\mathbf{x} \mid \mathbf{x}_1) \approx \delta(\mathbf{x} - \mathbf{x}_1)$. The marginal probability path is defined by:

$$p_t(\mathbf{x}) = \int_{\mathbb{R}^d} p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1 \tag{A.2}$$

Differentiating Equation A.2 with respect to time t:

$$\begin{aligned}\frac{\partial p_t(\mathbf{x})}{\partial t} &= \int_{\mathbb{R}^d} \frac{\partial p_t(\mathbf{x} \mid \mathbf{x}_1)}{\partial t} q(\mathbf{x}_1) d\mathbf{x}_1 \\ &= -\int_{\mathbb{R}^d} \nabla \cdot \left(p_t(\mathbf{x} \mid \mathbf{x}_1)\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\right) q(\mathbf{x}_1) d\mathbf{x}_1 \\ &= -\nabla \cdot \left(\int_{\mathbb{R}^d} p_t(\mathbf{x} \mid \mathbf{x}_1)\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1\right) \end{aligned} \tag{A.3}$$

Comparing Equation A.3 with the marginal continuity equation Equation A.1, the vector field $\mathbf{v}_t(\mathbf{x})$ is uniquely determined by:

$$p_t(\mathbf{x})\mathbf{v}_t(\mathbf{x}) = \int_{\mathbb{R}^d} p_t(\mathbf{x} \mid \mathbf{x}_1)\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1 \tag{A.4}$$

Dividing both sides by $p_t(\mathbf{x})$ yields:

$$\mathbf{v}_t(\mathbf{x}) = \int_{\mathbb{R}^d} \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1) \frac{p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1)}{p_t(\mathbf{x})} d\mathbf{x}_1 = \mathbb{E}_{\mathbf{x}_1 \sim q(\mathbf{x}_1 \mid \mathbf{x})} \left[\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\right] \tag{A.5}$$

A.1.2 Gradient Equivalence of the Loss Objective

The ideal marginal regression loss is given by:

$$\mathcal{L}_{\text{Marginal}}(\theta) = \mathbb{E}_{t \sim U[0,1], \mathbf{x} \sim p_t(\mathbf{x})} \left\|\mathbf{v}_\theta(\mathbf{x}, t) - \mathbf{v}_t(\mathbf{x})\right\|^2 \tag{A.6}$$

Expanding the squared L_2 norm:

$$\left\|\mathbf{v}_\theta(\mathbf{x}, t) - \mathbf{v}_t(\mathbf{x})\right\|^2 = \left\|\mathbf{v}_\theta(\mathbf{x}, t)\right\|^2 - 2\langle \mathbf{v}_\theta(\mathbf{x}, t), \mathbf{v}_t(\mathbf{x})\rangle + \left\|\mathbf{v}_t(\mathbf{x})\right\|^2 \tag{A.7}$$

Now consider the Conditional Flow Matching (CFM) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, \mathbf{x}_1 \sim q(\mathbf{x}_1), \mathbf{x} \sim p_t(\mathbf{x} \mid \mathbf{x}_1)} \left\|\mathbf{v}_\theta(\mathbf{x}, t) - \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\right\|^2 \tag{A.8}$$

Expanding the expectation:

$$\begin{aligned}\mathcal{L}_{\text{CFM}}(\theta) = &\int_0^1 \int\int \left\|\mathbf{v}_\theta(\mathbf{x}, t)\right\|^2 p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x} d\mathbf{x}_1 dt \\ &- 2\int_0^1 \int\int \langle \mathbf{v}_\theta(\mathbf{x}, t), \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\rangle p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x} d\mathbf{x}_1 dt \\ &+ C_1 \end{aligned} \tag{A.9}$$

where $C_1 = \mathbb{E}[\|\mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\|^2]$ is independent of the model parameters θ.

Applying Equation A.4 to the second term:

$$\begin{aligned}\int\int \langle \mathbf{v}_\theta(\mathbf{x},t), \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1)\rangle p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1 d\mathbf{x} &= \int \langle \mathbf{v}_\theta(\mathbf{x},t), \int \mathbf{v}_t(\mathbf{x} \mid \mathbf{x}_1) p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1 \rangle d\mathbf{x} \\ &= \int \langle \mathbf{v}_\theta(\mathbf{x},t), p_t(\mathbf{x}) \mathbf{v}_t(\mathbf{x}) \rangle d\mathbf{x} \\ &= \mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})} \left[\langle \mathbf{v}_\theta(\mathbf{x},t), \mathbf{v}_t(\mathbf{x}) \rangle \right] \end{aligned} \tag{A.10}$$

Similarly, the first term evaluates to:

$$\int\int \|\mathbf{v}_\theta(\mathbf{x},t)\|^2 p_t(\mathbf{x} \mid \mathbf{x}_1) q(\mathbf{x}_1) d\mathbf{x}_1 d\mathbf{x} = \mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})} \left[\|\mathbf{v}_\theta(\mathbf{x},t)\|^2 \right] \tag{A.11}$$

Therefore:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathcal{L}_{\text{Marginal}}(\theta) + C(\mathbf{x}_1) \tag{A.12}$$

Consequently, their gradients with respect to network parameters θ are identically equal:

$$\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) \equiv \nabla_\theta \mathcal{L}_{\text{Marginal}}(\theta) \tag{A.13}$$

This confirms that optimizing the conditional objective over individual training samples yields parameter gradients identical to those of the marginal generative flow.

A.2 Analytical Formulation of the Inverse STFT with Window Overlap-Add

Let $\mathbf{Z}(k,m) \in \mathbb{C}$ denote the synthesized discrete complex Fourier matrix at frequency bin $k \in [0, N_{\text{fft}} - 1]$ and time frame $m \in \mathbb{Z}$. The one-dimensional Inverse Discrete Fourier Transform (IDFT) of each individual column frame is:

$$z_m(n) = \frac{1}{N_{\text{fft}}} \sum_{k=0}^{N_{\text{fft}}-1} \mathbf{Z}(k,m) e^{j\frac{2\pi}{N_{\text{fft}}}kn}, \quad n \in [0, N_{\text{fft}} - 1] \tag{A.14}$$

Each frame is weighted by a periodic synthesis Hanning window $w(n)$:

$$w(n) = 0.5 \left(1 - \cos\left(\frac{2\pi n}{N_{\text{fft}}}\right)\right), \quad n \in [0, N_{\text{fft}} - 1] \tag{A.15}$$

The time-domain output $y(n)$ is reconstructed via the Weighted Overlap-Add (WOLA) method with frame hop $H = 256$:

$$y(n) = \frac{\sum_{m=-\infty}^{\infty} z_m(n - mH) w(n - mH)}{\sum_{m=-\infty}^{\infty} w^2(n - mH)} \tag{A.16}$$

When the synthesis window satisfies the Principle of Perfect Reconstruction:

$$\sum_{m=-\infty}^{\infty} w^2(n - mH) = C_{\text{norm}} \quad \forall\, n \tag{A.17}$$

the denominator in Equation A.16 evaluates to a constant scalar, allowing the reconstruction to be computed in linear time $O(M \cdot N_{\text{fft}} \log N_{\text{fft}})$ using ARM NEON vector instructions.

Appendix B

Multilingual Phonetic Evaluation Prompts Dataset

B.1 Dataset Architecture and Stratification Methodology

To evaluate voice cloning fidelity, articulation clarity, and edge execution throughput under controlled experimental conditions, we curated the standardized dataset `prompts.json`. The corpus comprises 120 phonetically balanced conversational sentences distributed across Tamil, Telugu, and Hindi (40 phrases per language).

To quantify temporal scaling dynamics and verify our syllabic duration formula across diverse utterance lengths, the corpus is stratified into four standardized duration tiers:

1. **Short Tier** ($\sim$ 4–7 words): Everyday inquiries, greetings, and brief conversational exchanges.
2. **Medium Tier** ($\sim$ 12–18 words): Single-clause sentences conveying social activities and observations.
3. **Midlong Tier** ($\sim$ 20–25 words): Compound descriptive sentences containing coordinate clauses.
4. **Long Tier** ($\sim$ 28–35 words): Complex narrative sentences designed to stress-test attention mechanisms and RAM allocations.

B.2 Telugu Dataset Inventory

Table B.1 documents representative Telugu evaluation prompts across standardized duration tiers, displaying the native Telugu script along with English semantic translations.

Table B.1: Curated Telugu Evaluation Prompts across Standardized Tiers

Tier	**Target Utterance (Telugu Script)**	**English Semantic Translation**
Short 1	నువ్వు భోజనం చేశావా?	Have you had your meal?
Short 2	ఈరోజు వాతావరణం చాలా బాగుంది.	The weather is very pleasant today.
Short 3	రేపు సాయంత్రం కలుద్దామా?	Shall we meet tomorrow evening?
Short 4	నాకు కాసిన్ని నీళ్లు కావాలి.	I need some water, please.
Short 5	ఇంటికి వెళ్లే సైం అయ్యింది.	It is time to head back home.
Short 6	ఈ పుస్తకం చదవడం నాకు నచ్చింది.	I enjoyed reading this book.
Short 7	అందరూ బాగున్నారా?	Is everyone doing well?
Medium 1	చాలా రోజుల తర్వాత నిన్ను చూడటం చాలా సంతోషంగా ఉంది.	I am glad to see you after such a long time.
Medium 2	ఈరోజు ఉదయం నుండి చల్లగా ఉంది, వర్షం వచ్చేలా ఉంది.	It has been cool since morning; it looks like rain.
Medium 3	మా ఊర్లో పండగ సైంలో అందరూ చాలా ఉత్సాహంగా ఉంటారు.	Everyone in our village is energetic during festival time.
Medium 4	పిల్లలు తోటలో పువ్వులు చూస్తూ సంతోషంగా ఆడుకుంటున్నారు.	Children are happily playing in the garden admiring flowers.
Midlong 1	ప్రతిరోజూ ఉదయం లేచి కొంచెం సేపు నడిస్తే రోజంతా చురుగ్గా, ఆరోగ్యంగా ఉంటుంది.	Walking every morning keeps one active and healthy throughout the day.
Midlong 2	మా ఊరి చెరువు దగ్గర పచ్చని పొలాలు, తాటి చెట్లు, పక్షుల అరుపులు చూస్తుంటే మనసుకి చాలా బాగుంటుంది.	The green fields, palm trees, and birds by our village pond are delightful.
Long 1	మన కుటుంబంలో అందరం కలిసి పండగలు చేసుకోవడం వల్ల మన మధ్య అనుబంధం ఇంకా బలంగా అవుతుంది.	Celebrating festivals together strengthens our family bonds.
Long 2	రైతులు ఎండైనా వానైనా లెక్క చేయకుండా పొలంలో కష్టపడి పని చేస్తారు కాబట్టే మనందరికీ తినడానికి తిండి దొరుకుతుంది.	Because farmers work tirelessly through sun and rain, we have food to eat.

B.3 Tamil Dataset Inventory

Table B.2 documents representative Tamil evaluation prompts across standardized duration tiers, displaying the native Tamil script along with English semantic translations.

Table B.2: Curated Tamil Evaluation Prompts across Standardized Tiers

Tier	**Target Utterance (Tamil Script)**	**English Semantic Translation**
Short 1	நீ நலமா இருக்கியா?	Are you doing well?
Short 2	இன்று மாலை கடற்கரைக்குப் போவோமா?	Shall we go to the beach this evening?
Short 3	எனக்கு கொஞ்சம் தண்ணி வேணும்.	I need some drinking water.
Short 4	இந்த புத்தகம் ரொம்ப நல்லா இருக்கு.	This book is very good.
Short 5	நேரத்த வீணடிக்க கூடாது.	We should not waste our time.
Short 6	வீட்டுக்குப் போக நேரமாச்சு.	It is time to head back home.
Short 7	உன்னைப் பார்த்ததுல ரொம்ப சந்தோஷம்.	I am very glad to see you.
Medium 1	நீண்ட நாளுக்கு அப்புறம் உன் குரலக் கேட்டது ரொம்ப சந்தோஷமா இருக்கு.	Hearing your voice after a long time brings great joy.
Medium 2	இன்று காலையிலிருந்து குளிர் காத்து வீசுது, மழை வரலாம் போல.	A cool breeze has been blowing since morning; it looks like rain.
Medium 3	தோட்டத்துல பூக்கள் மலர்ந்து இருக்கு, வீடு முழுக்க வாசனை.	Flowers have bloomed in the garden, filling the house with fragrance.
Medium 4	குழந்தைங்க மாலையில ஒண்ணா சேர்ந்து விளையாடுறது பார்க்க நல்லா இருக்கு.	Watching children play together in the evening is lovely.
Midlong 1	தினமும் காலையில எழுந்து கொஞ்ச நேரம் நடந்தா, நாள் முழுக்க உடம்பு லேசா இருக்கும்.	Walking a bit every morning keeps the body feeling refreshed throughout the day.
Midlong 2	எங்க ஊரு வயல் வெளி, பனை மரங்கள், பறவைகளோட சத்தம் எல்லாம் ரொம்ப அழகா இருக்கும்.	The open fields, palm trees, and birds in our village are scenic.
Long 1	நம்ம குடும்பத்துல எல்லாரும் சேர்ந்து பண்டிகை கொண்டாடுறது நம்ம பந்தத்த இன்னும் பலமா ஆக்கும்.	Celebrating festivals together strengthens our family ties.
Long 2	விவசாயிகள் வெயில், மழை பாக்காம இரவு பகலா உழைக்கிறதால தான் நமக்கு சாப்பாடு கிடைக்குது, அவங்கள மதிக்கணும்.	Because farmers labor day and night regardless of heat or rain, we have food to eat.

B.4 Hindi Dataset Inventory

Table B.3 documents representative Hindi evaluation prompts across standardized duration tiers, displaying the native Devanagari script along with English semantic translations.

Table B.3: Curated Hindi Evaluation Prompts across Standardized Tiers

Tier	**Target Utterance (Devanagari Script)**	**English Semantic Translation**
Short 1	तुम कैसे हो?	How are you doing?
Short 2	आज शाम घूमने चलें?	Shall we go out for a walk this evening?
Short 3	मुझे थोड़ा पानी चाहिए।	I would like some water.
Short 4	यह किताब बहुत अच्छी है।	This book is very good.
Short 5	समय बर्बाद नहीं करना चाहिए।	We should not waste time.
Short 6	घर जाने का समय हो गया।	It is time to head back home.
Short 7	तुमसे मिलकर अच्छा लगा।	It was pleasant meeting you.
Medium 1	इतने दिनों बाद तुमसे बात करके बहुत अच्छा लग रहा है।	It feels wonderful speaking with you after so many days.
Medium 2	आज सुबह से ठंडी हवा चल रही है, लगता है बारिश आएगी।	Cool breeze has been blowing since morning; it seems rain is coming.
Medium 3	बगीचे में फूल खिले हैं, पूरा घर महक रहा है।	Flowers are blooming in the garden, and the whole house smells fresh.
Medium 4	शाम को बच्चों को साथ खेलते देखना अच्छा लगता है।	Watching children play together in the evening is heartwarming.
Midlong 1	रोज सुबह उठकर थोड़ी देर टहलने से पूरे दिन शरीर हल्का और तरोताज़ा रहता है।	Walking a little every morning keeps the body active and fresh all day.
Midlong 2	गाँव के खेत, पेड़ और पक्षियों की आवाज़ें मन को बहुत सुकून देती हैं।	The village fields, trees, and chirping birds bring peace of mind.
Long 1	हमारे घर में सब मिलकर त्योहार मनाते हैं, इससे हमारा आपसी रिश्ता और मजबूत होता है।	Celebrating festivals together as a family deepens our mutual relationships.
Long 2	किसान धूप और बारिश की परवाह किए बिना दिन-रात मेहनत करते हैं, इसी वजह से हमें खाना मिलता है, इसलिए उनका सम्मान करना चाहिए।	Because farmers work tirelessly day and night regardless of sun or rain, we have food.

Bibliography

[1] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *International Conference on Machine Learning (ICML)*, pages 5530–5540, 2021.

[2] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 17022–17033, 2020.

[3] Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rasmus Moritz, Mary Williamson, Vimal Manohar, Yossi Adi, Jay Mahadeokar, and Wei-Ning Hu. Voicebox: Text-guided multilingual universal speech generation at scale. *arXiv preprint arXiv:2306.15687*, 2023.

[4] Sefik Emre Eskimez, Xiaofei Wang, Manthan Thakker, Canrun Li, Chung-Hsien Tsai, Zhen Xiao, Hemin Yang, Zirun Zhu, Min Tang, Sheng Zhao, and Naoyuki Kanda. E2 TTS: Fully non-autoregressive zero-shot text-to-speech. *arXiv preprint arXiv:2406.18009*, 2024.

[5] Yushen Chen, Zhikang Niu, Ziyang Ma, Keqi Liu, Guanrou Zhao, Kai Shen, Chenpeng Lin, Zhijun Wang, Xie Chen, and Heng Lu. F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*, 2024.

[6] Edress Casanova, Kelly Weber, Christopher Shulby, et al. XTTS: Massively multilingual voice cloning with latent diffusion and autoregressive priors. *arXiv preprint arXiv:2406.04904*, 2024.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:6840–6851, 2020.

[9] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[10] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nicklas, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[11] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023.

[12] Hubert Siuzdak. Vocos: Closing the gap between neural vocoders and fourier-based audio synthesis. *arXiv preprint arXiv:2306.00814*, 2023.

[13] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[14] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[15] Zhen Ye, Wei Xue, Xu Tan, Qifeng Chen, and Yike Guo. FlashSpeech: Efficient zero-shot speech synthesis with consistency models. *arXiv preprint arXiv:2404.14700*, 2024.

[16] Tahir Javed, Sumanth Doddapaneni, Abhigyan Raman, Kaushal Seshadri, Pratyush Kumar, and Mitesh M. Khapra. Indicvoices: Towards building an inclusive multilingual speech dataset for indian languages. *arXiv preprint arXiv:2403.07639*, 2024.

[17] S. V. Praveen, Srija Anand, Soma Siddhartha, and Mitesh M. Khapra. Phir hera fairy: An english fairytaler is a strong faker of fluent speech in low-resource indian languages. *arXiv preprint arXiv:2505.20693*, 2025.

[18] Next-gen Kaldi Team. Sherpa-onnx: Next-gen kaldi on-device speech recognition and synthesis. `https://github.com/k2-fsa/sherpa-onnx`, 2023.

[19] Microsoft Corporation. ONNX Runtime Mobile: High performance machine learning on mobile and edge devices. `https://onnxruntime.ai/docs/reference/mobile/`, 2023.

[20] Google AI Edge Team. Litert: Unified on-device machine learning framework. `https://ai.google.dev/edge/litert`, 2024.

[21] Google AI Edge Team. Litert torch: High performance pytorch on-device deployment. `https://github.com/google-ai-edge/litert-torch`, 2024.

[22] Qualcomm Technologies, Inc. Qualcomm AI engine direct (QNN) software development kit. `https://developer.qualcomm.com/software/qualcomm-ai-engine-direct-sdk`, 2024.

[23] MediaTek Inc. Mediatek NeuroPilot edge AI platform architecture and software toolkit. `https://neuropilot.mediatek.com/`, 2024.